

AIMS 5740

Generative Artificial Intelligence

(2026 Term 2)

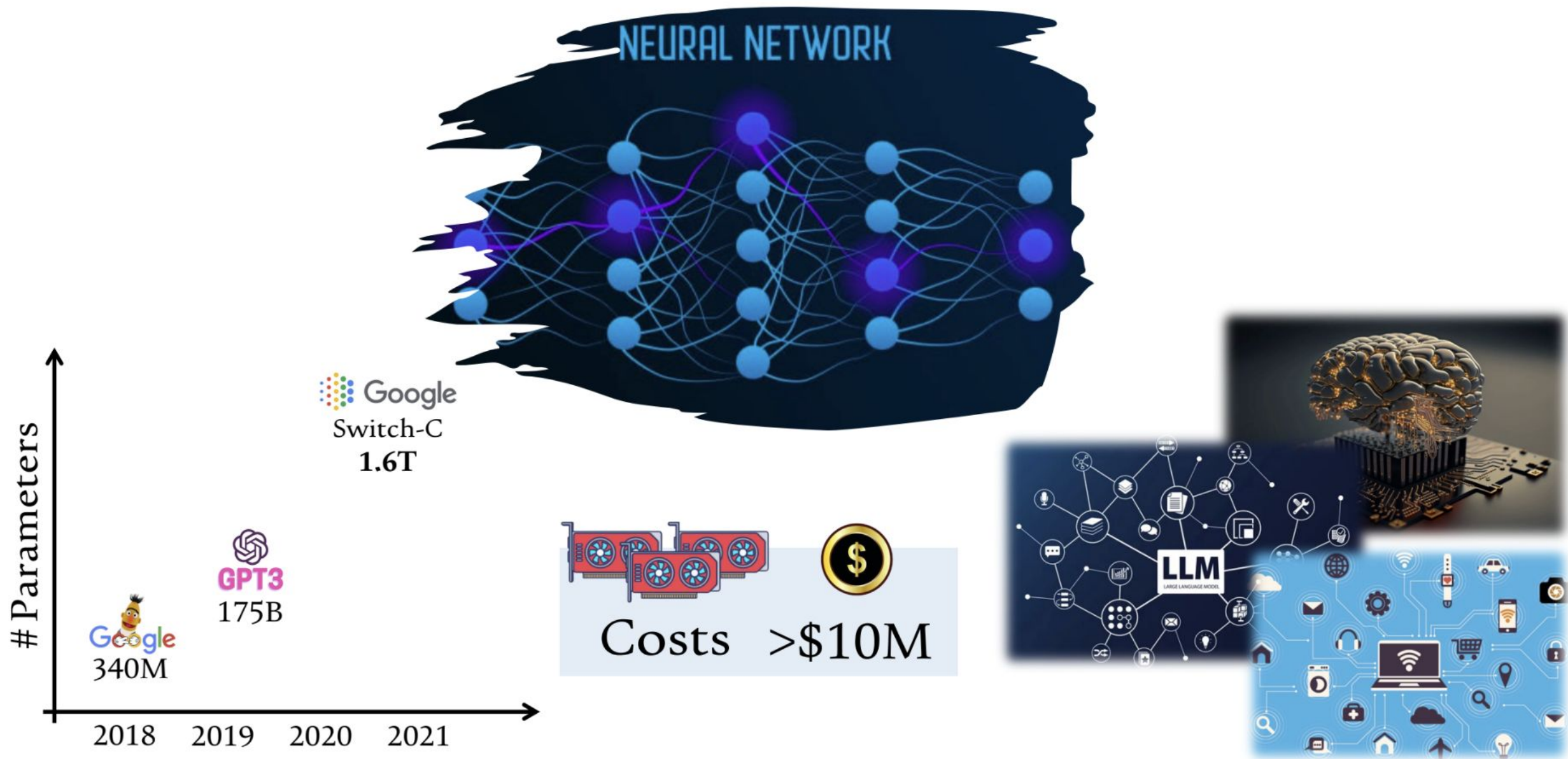


Computer Science & Engineering
The Chinese University of Hong Kong

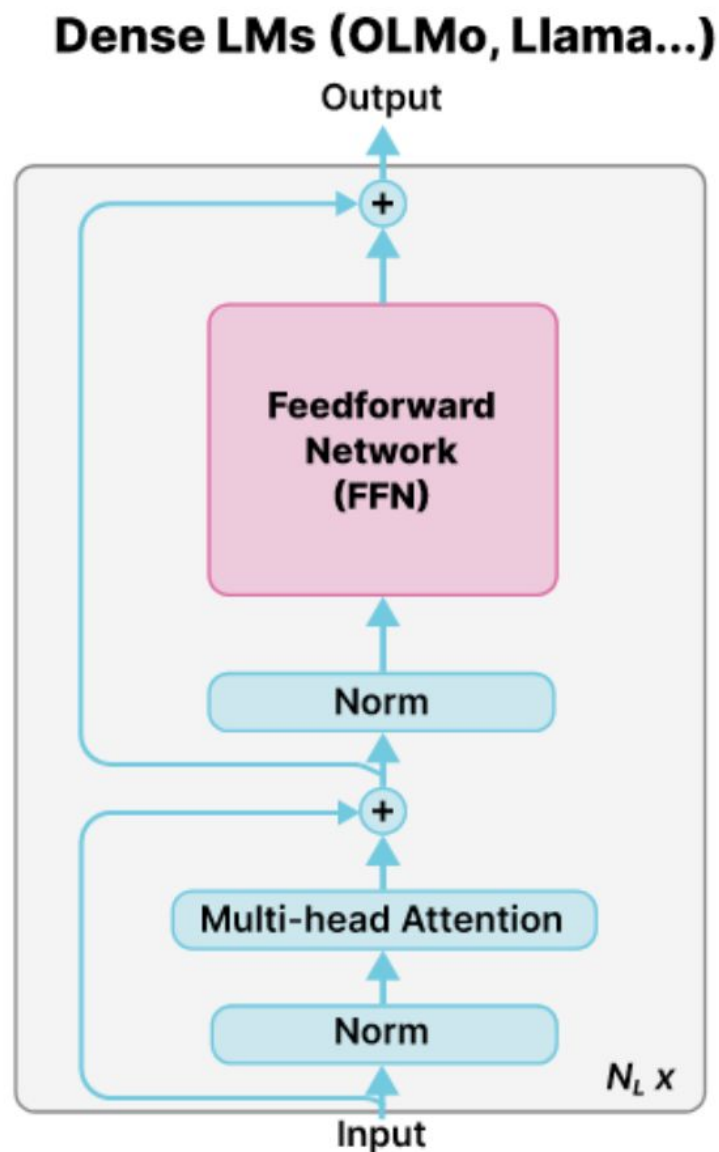
Announcement

- Our second assignment will be released on Mar. 18.
- The team information for the course project is ready.
- We will have a revision for the midterm exam on Mar. 11.

Current AI models are Gigantic



The Linear Layer in LLMs

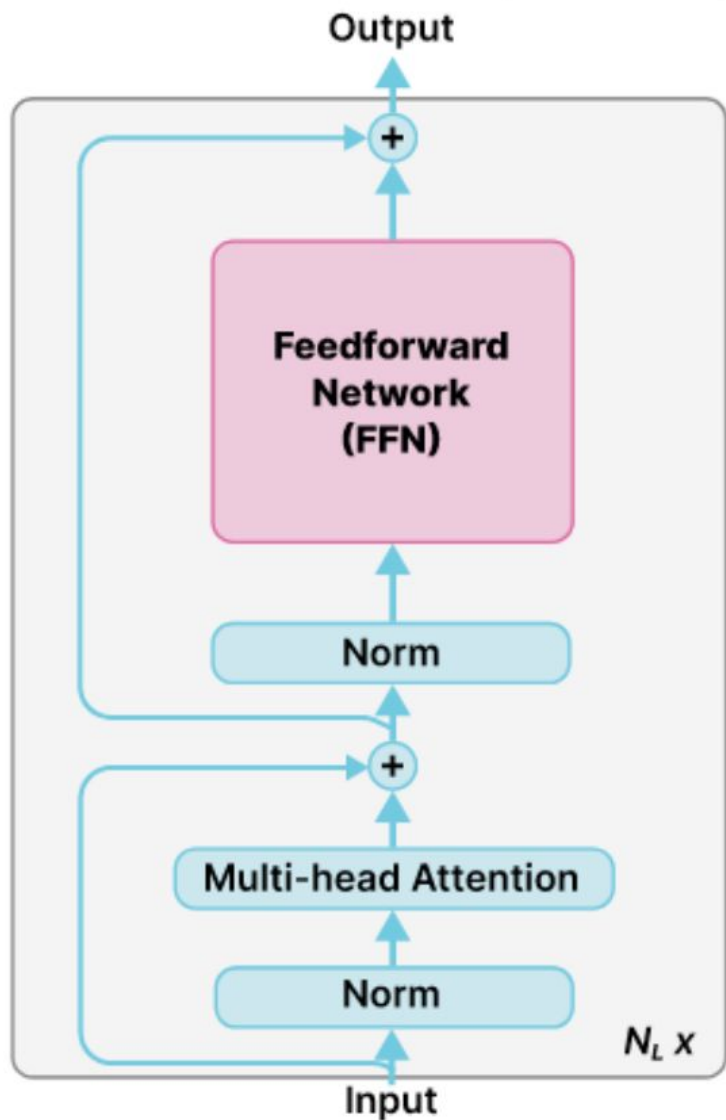


It is common for more than half of the parameters in a Transformer LLM to reside within the feed-forward neural network layers

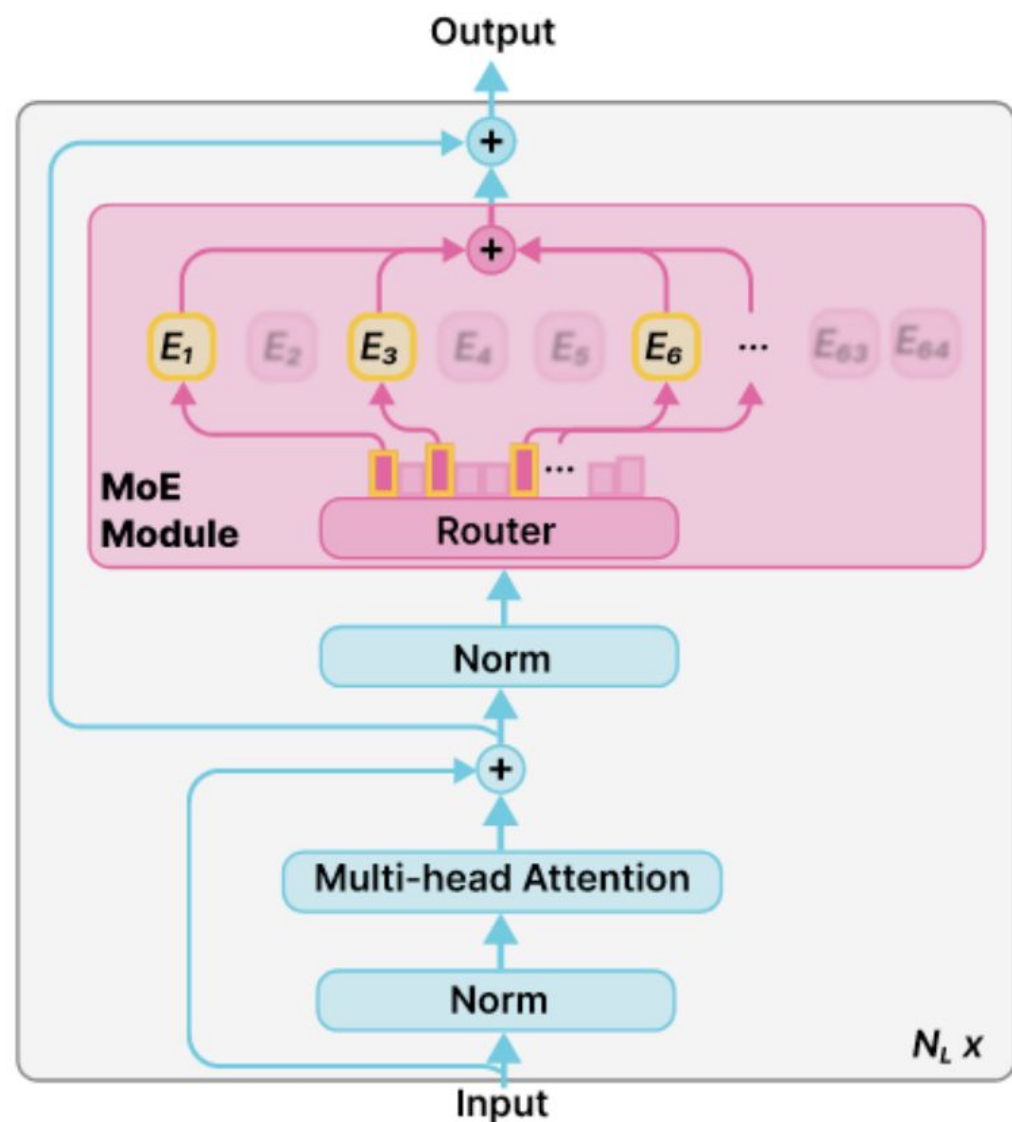
	GPT-3
vocab	50,257
d_model	12288
n_heads	96
n_layers	96
d_ff	49152
LayerNorm (B)	0.00
Embedding (B)	0.62
Attention (B)	57.99
Feed-forward (B)	115.97
Total (B)	174.57

Sparsely-Gated Mixture of Experts

Dense LMs (OLMo, Llama...)



OLMoE



Dense Mixture of Experts

A dense mixture of experts gives every expert a (non-zero) voice in the output

$$y = \sum_{i=1}^{N_e} G(\mathbf{x})_i E_i(\mathbf{x})$$

Dense MoE

- Dense Softmax

$$G(\mathbf{x}) = \text{softmax}(\mathbf{x} \cdot \mathbf{W}_g)$$

- Each expert is just a feed-forward network

$$E_i(\mathbf{x}) = \text{FFN}_{\text{ReLU}}(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

Sparsely-Gated Mixture of Experts

Sparsely-Gated MoEs were originally introduced for RNNs, but are generally applicable and now popular for Transformers

$$y = \sum_{i=1}^{N_e} G(\mathbf{x})_i E_i(\mathbf{x})$$

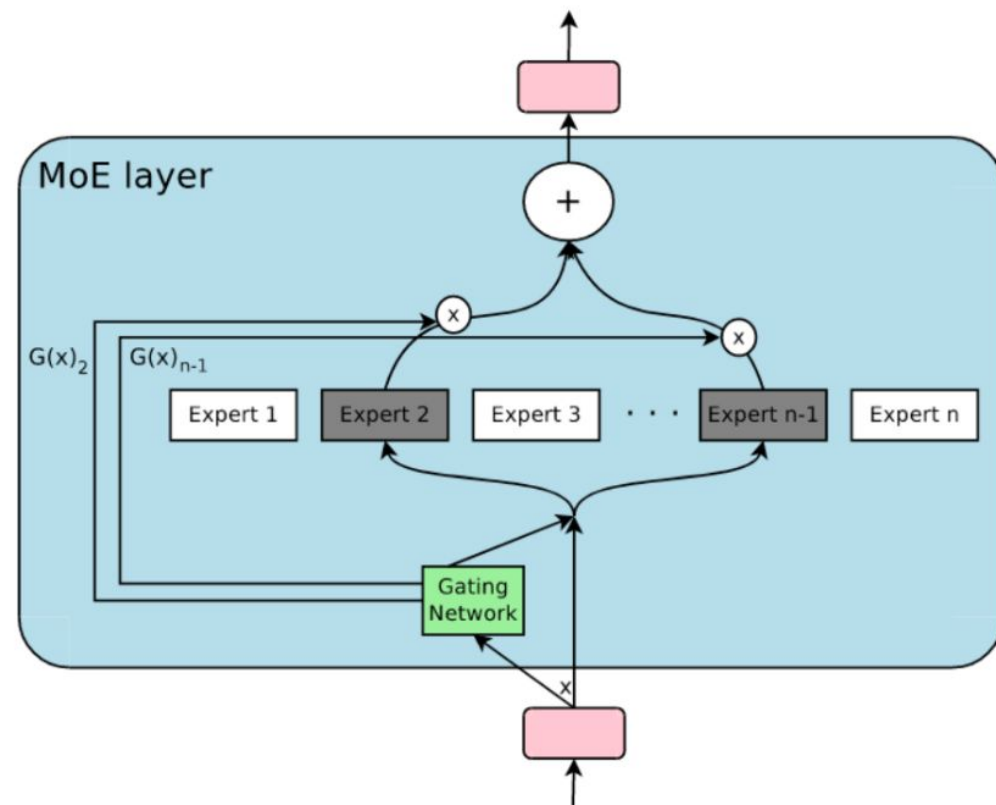
Sparse MoE

- Softmax over Top-K Gating

$$G(\mathbf{x}) = \text{softmax}(\text{topk}(\mathbf{x} \cdot \mathbf{W}_g + \mathbf{b}_g, k))$$

- Each expert is just a feed-forward network

$$E_i(\mathbf{x}) = \text{FFN}_{\text{ReLU}}(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$



Sparsely-Gated Mixture of Experts

Sparsely-Gated MoEs were originally introduced for RNNs, but are generally applicable and now popular for Transformers

$$y = \sum_{i=1}^{N_e} G(\mathbf{x})_i E_i(\mathbf{x})$$

Sparse MoE

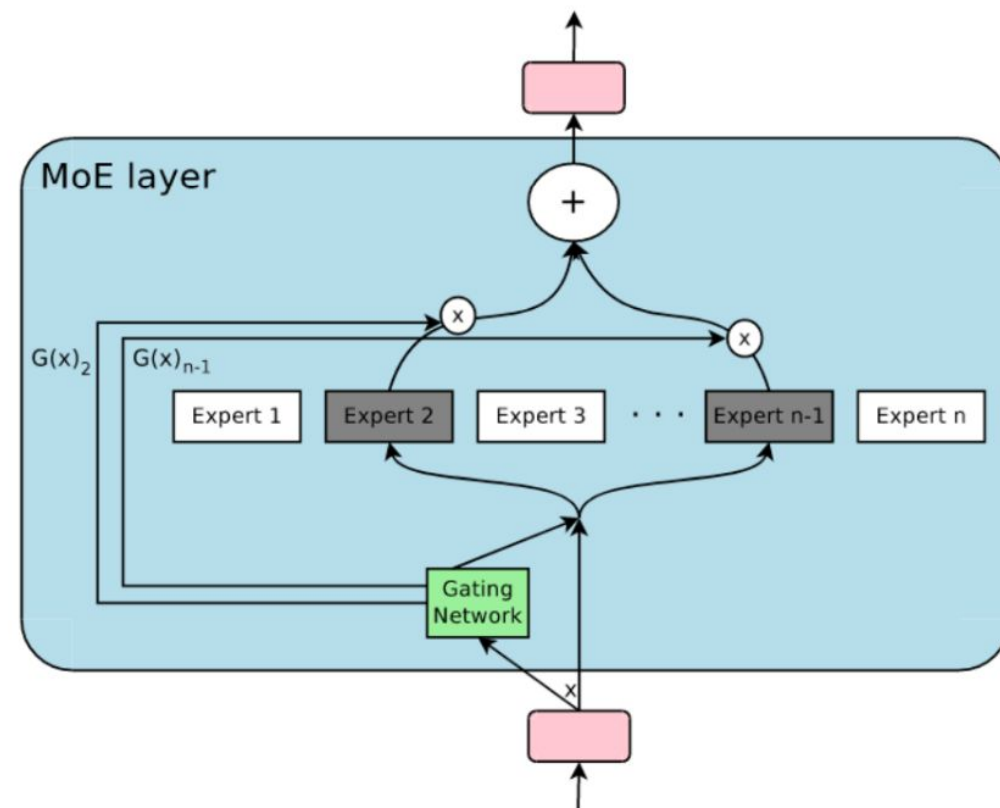
- Softmax over Top-K Gating

$$G(\mathbf{x}) = \text{softmax}(\text{topk}(\mathbf{x} \cdot \mathbf{W}_g + \mathbf{b}_g + \mathbf{r}_{\text{noise}}(\mathbf{x}), k))$$

$$\mathbf{r}_{\text{noise}}(\mathbf{x}) = N(\mathbf{0}, \mathbf{I}) \cdot \text{sigma}(\mathbf{x} \mathbf{W}_{\text{noise}} + \mathbf{b}_{\text{noise}})$$

- Each expert is just a feed-forward network

$$E_i(\mathbf{x}) = \text{FFN}_{\text{ReLU}}(\mathbf{x}) = \text{ReLU}(\mathbf{x} \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$$



Sparsely-Gated Mixture of Experts

Sparsely-Gated MoEs were originally introduced by DeepMind and are generally applicable and now popular for

$$y = \sum_{i=1}^{N_e} G(\mathbf{x})_i E_i(\mathbf{x})$$

Mixtral

- Softmax over Top-K Gating

$$G(\mathbf{x}) = \text{softmax}(\text{topk}(\mathbf{x} \cdot \mathbf{W}_g + \mathbf{b}_g + \mathbf{r}_{\text{noise}}(\mathbf{x}), k))$$

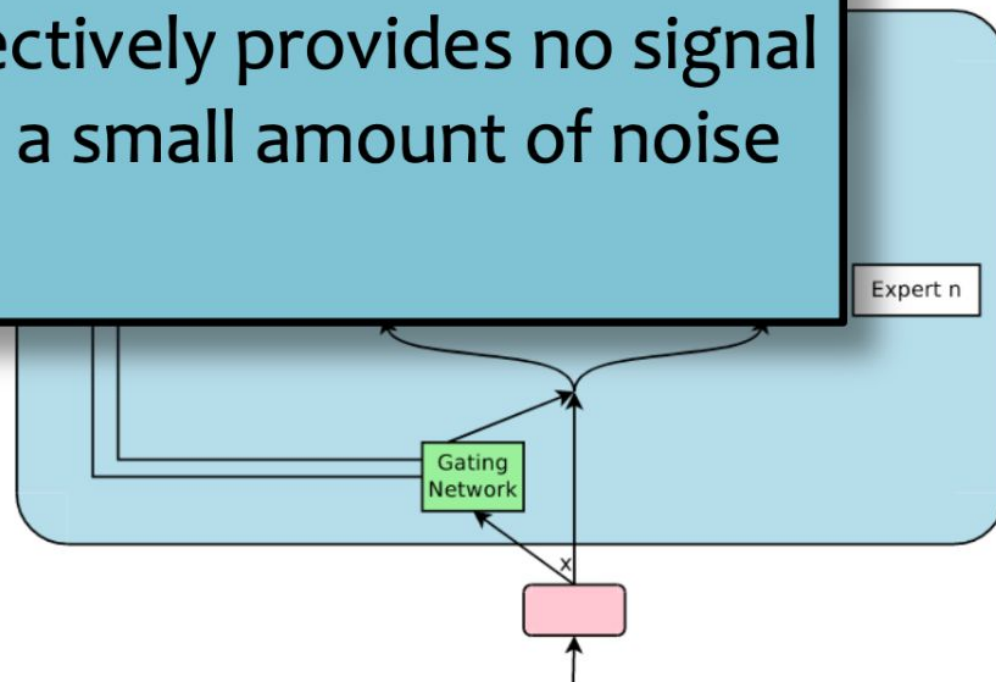
$$\mathbf{r}_{\text{noise}}(\mathbf{x}) = N(\mathbf{0}, \mathbf{I}) \cdot \text{sigma}(\mathbf{x} \mathbf{W}_{\text{noise}} + \mathbf{b}_{\text{noise}})$$

- Each expert is just a feed-forward network

$$E_i(\mathbf{x}) = \text{SwiGLU}(\mathbf{x}) = \text{Swish}(\mathbf{x} \mathbf{W} + \mathbf{b}) \otimes (\mathbf{x} \mathbf{V} + \mathbf{c})$$

Initialization:

- We initialize $\mathbf{W}_g$ and $\mathbf{W}_{\text{noise}}$ to all zeros
- Effectively provides no signal and a small amount of noise



Balancing Expert Utilization in a MoE

- **Problem:** left unchecked, the expert gate tends to concentrate on a small number of experts that are popular early in training
- **Solution:**
 - Add two regularizers to the loss
 - *Load Balance Term:* encourages distributed load over the experts within each batch
 - *Router Z-loss:* penalizes large logits to the router to stabilize training

(this is the approach used by OLMoE, but lots of variants exist)

$$\mathcal{L} = \mathcal{L}_{CE} + \alpha \mathcal{L}_{LB} + \beta \mathcal{L}_{RZ}$$

$$\mathcal{L}_{LB} = N_e \sum_{i=1}^{N_e} f_i P_i$$

f_i = fraction of tokens routed to expert i

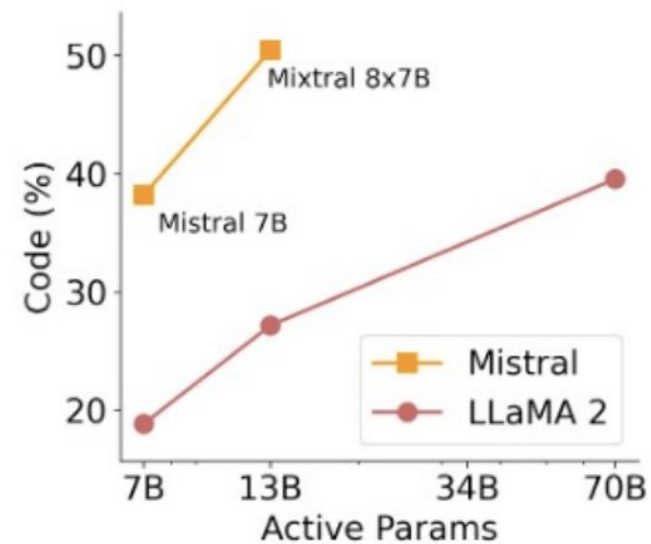
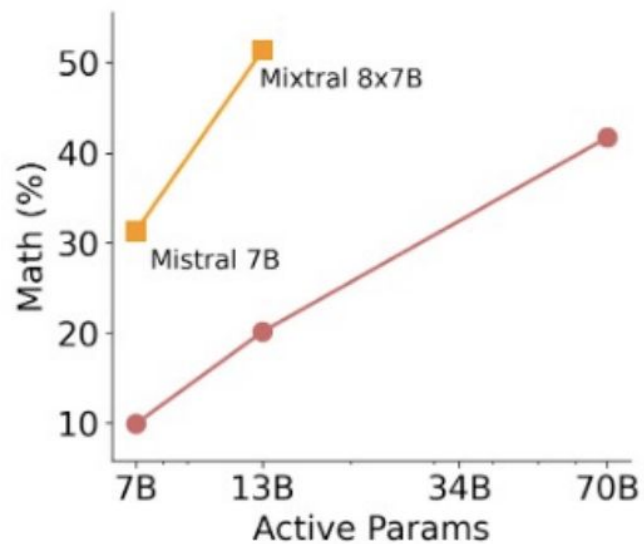
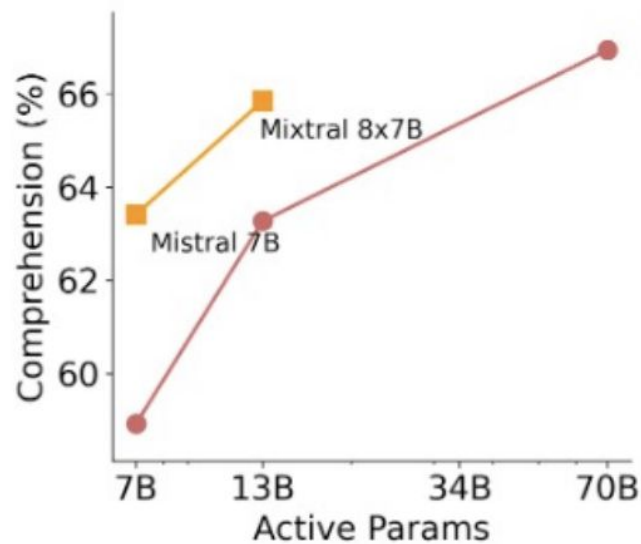
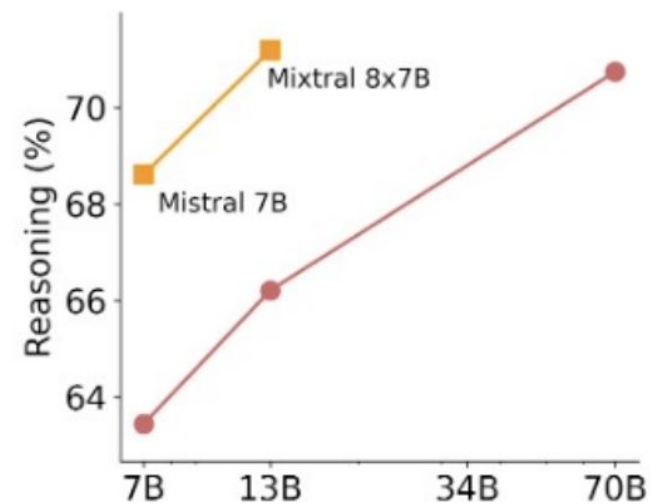
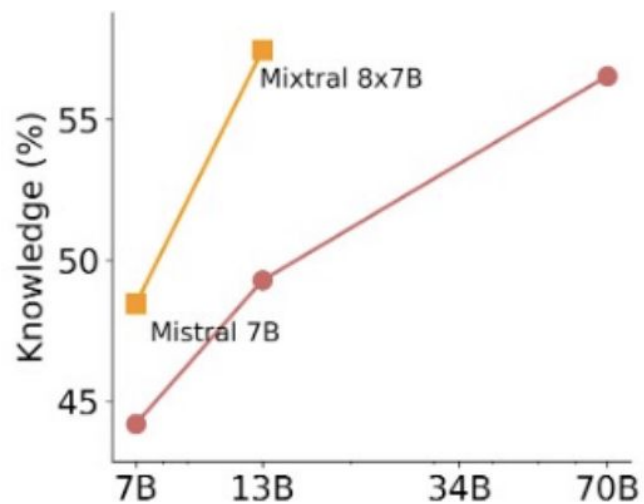
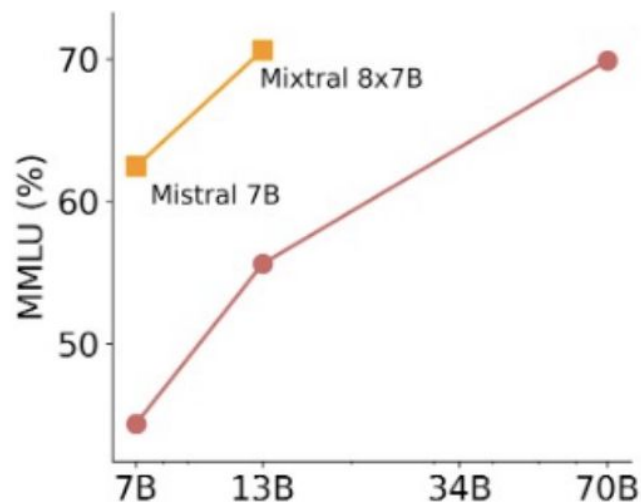
P_i = probability to E_i in current batch

$$\mathcal{L}_{RZ} = \frac{1}{N_b} \sum_{b=1}^{N_b} \left(\log \sum_{d=1}^D \exp(x_d^{(b)}) \right)$$

| Active Parameters

- In a transformer with MoE layers, we typically choose the number k for the top- k to be rather small
 - Mixtral: $k = 2$, $N_e = 8$
 - OlMoE: $k=8$, $N_e = 64$
- The number of **active parameters** is the count of parameters that are selected for computation by the router
- For an MoE (roughly)
 - GPU memory requirement $\propto$ # of total parameters
 - FLOPS computation requirement $\propto$ # of active parameters

Mixtral vs. Llama-2



Performance vs. Cost

MoEs provide a nice tradeoff between performance and FLOPS cost

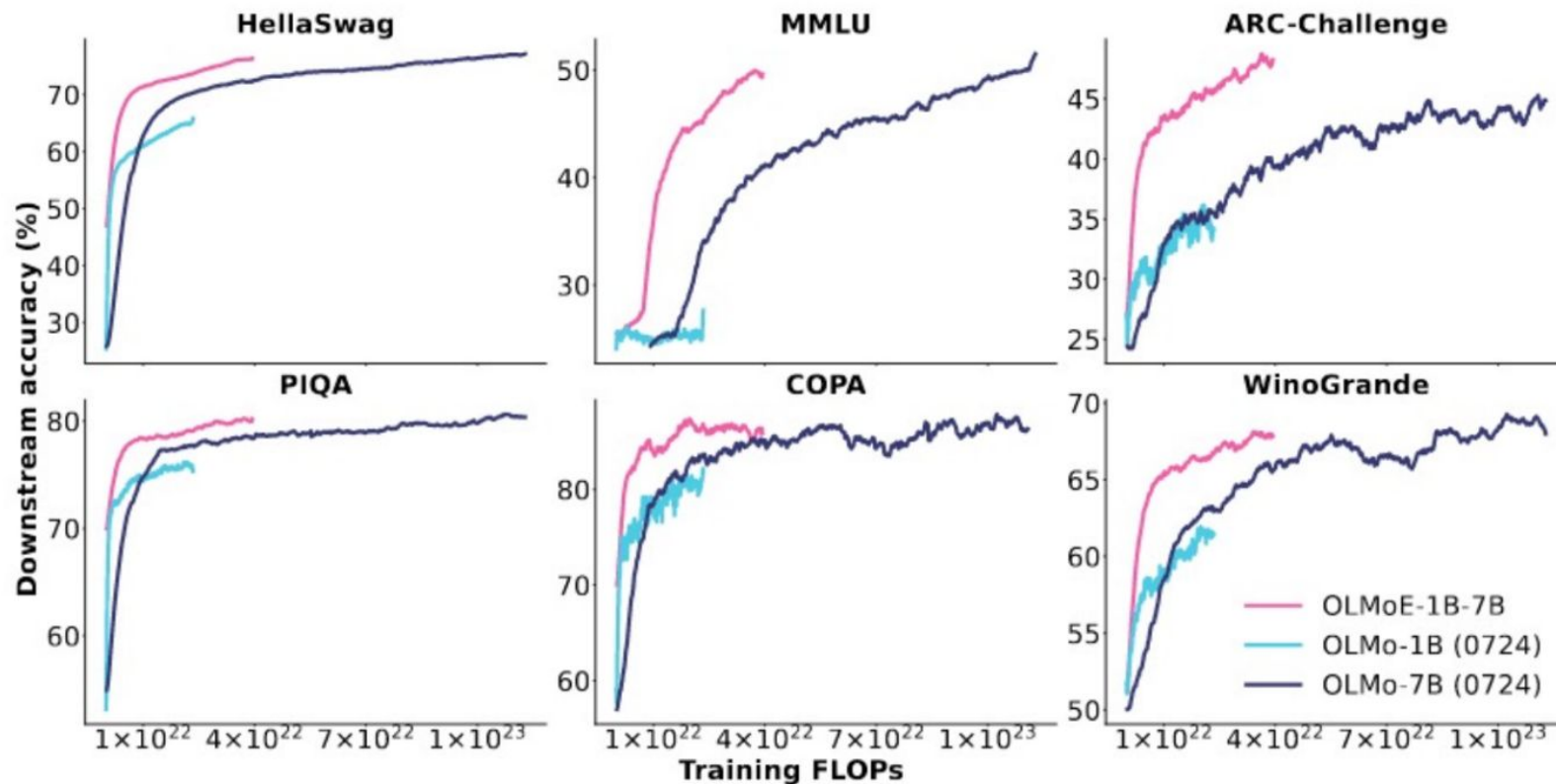


Figure 3: Evaluation of OLMoE-1B-7B and the current best OLMo models during pretraining. OLMoE-1B-7B differs from the OLMo models in its MoE architecture, several training hyperparameters, and its training dataset, see §2. A version of this plot with tokens as the x-axis and markers where annealing starts is in Appendix E. More results, logs, and configurations: <https://wandb.ai/ai2-llm/olmoe/reports/Plot-OLMoE-1B-7B-vs-OLMo-7B-vs-OLMo-1B--Vmlldzo40TcyMjEz>

Performance vs. Cost

MoEs provide a nice tradeoff between performance and FLOPS cost

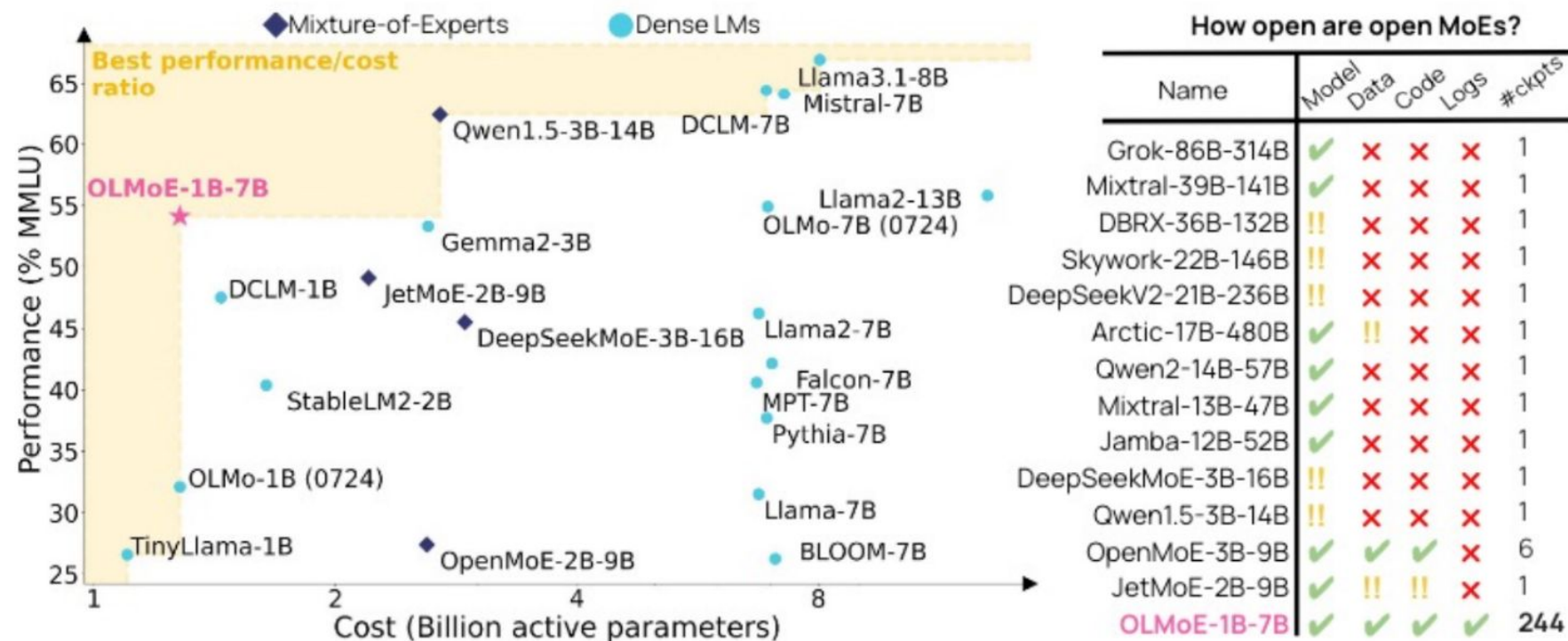
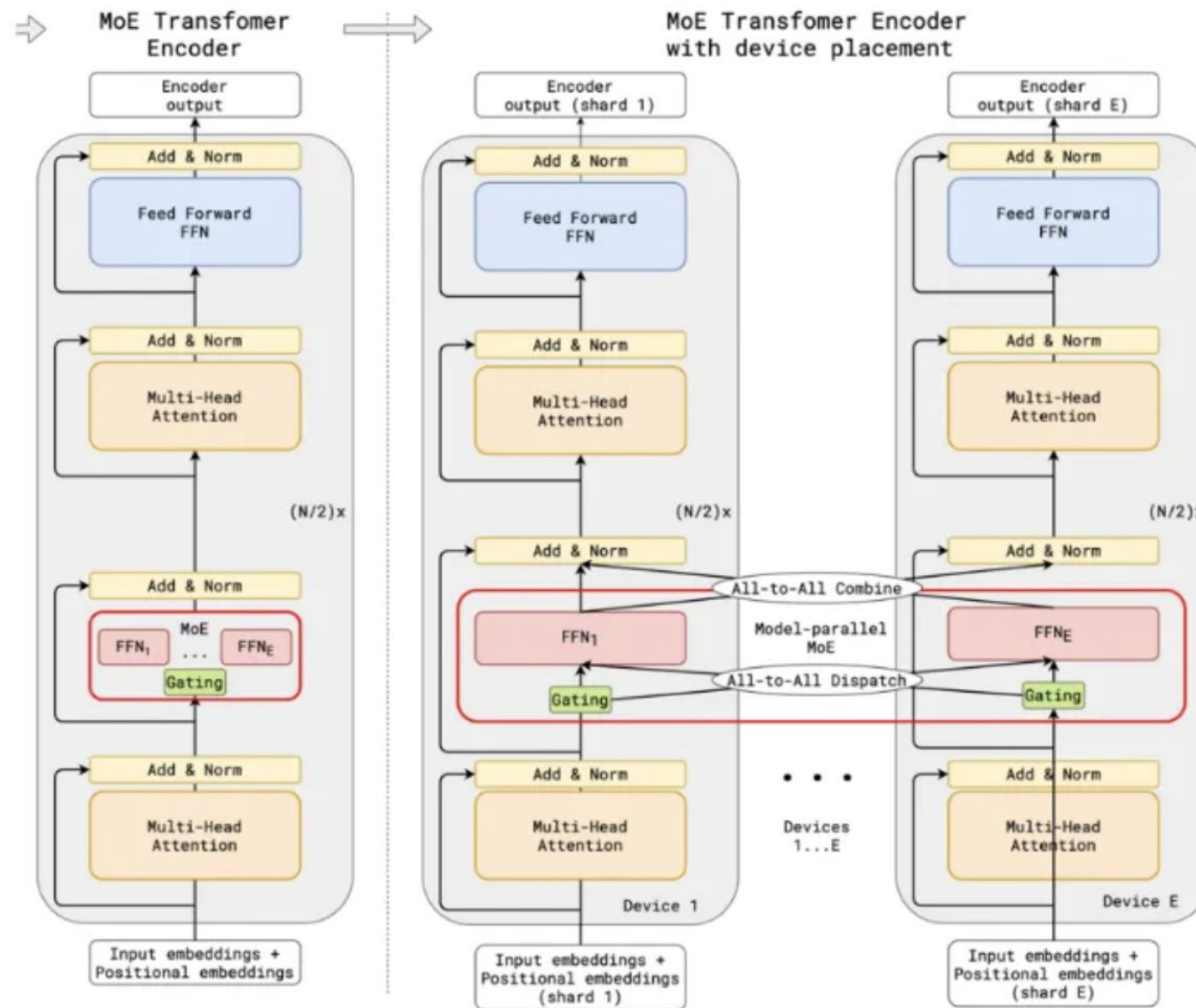


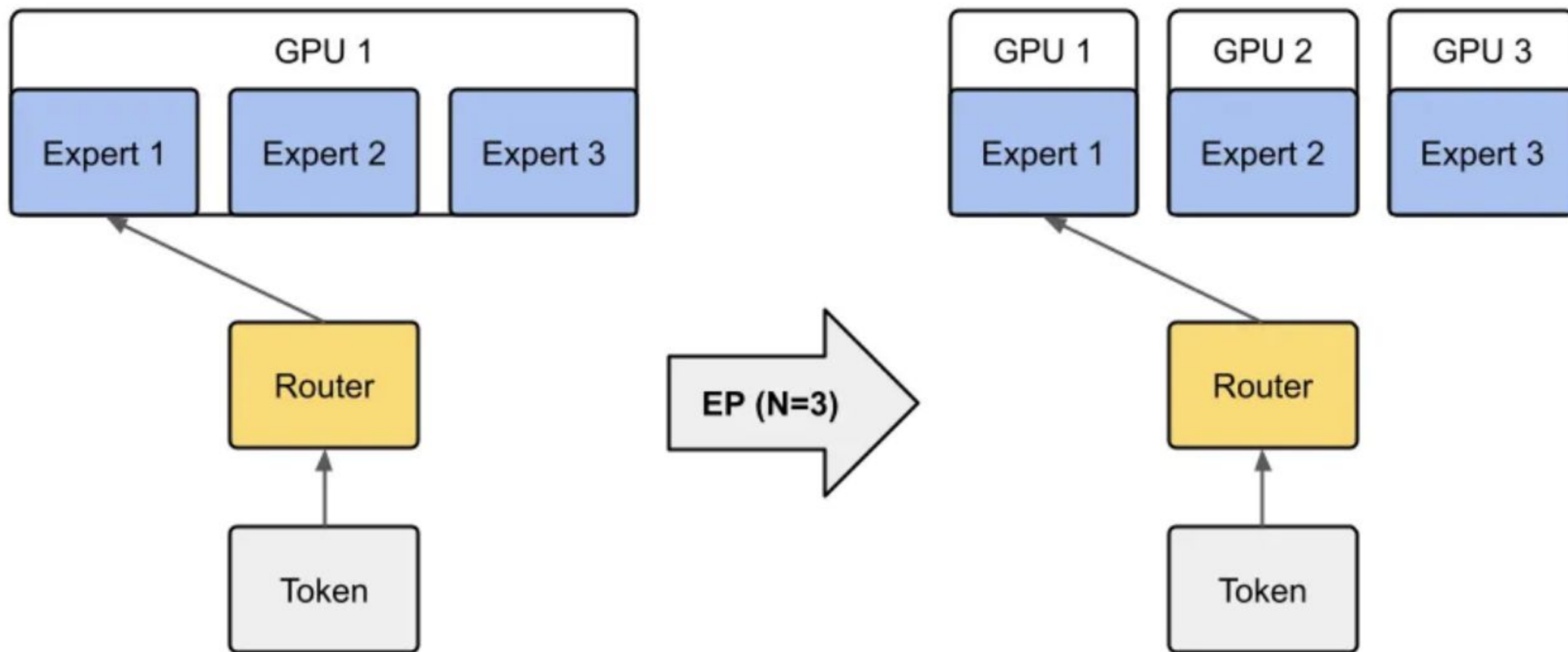
Figure 1: **Performance, cost, and degree of openness of open MoE and dense LMs.** Model names contain rounded parameter counts: model-active-total for MoEs and model-total for dense LMs. #ckpts is the number of intermediate checkpoints available. We highlight MMLU as a summary of overall performance; see §3 for more results. **OLMoE-1B-7B** performs best among models with similar active parameter counts and is the most open MoE.

MoEs and Parallelism (1)



MoEs and Parallelism (2)

Expert Parallelism applied on Mixture-of-Experts.



MoE - Fine-tuning

- Instruction-tuning is better than fine-tuning: training with mixed prompt settings (zero-shot, few-shot, and chain-of-thought;
- Sparse models have performed remarkably well in the regime of large datasets, but have sometimes performed poorly when fine tuning data is limited;
- In general, MoE model performance scales better with respect to the number of tasks rather than the number of experts.

Flan-MoE

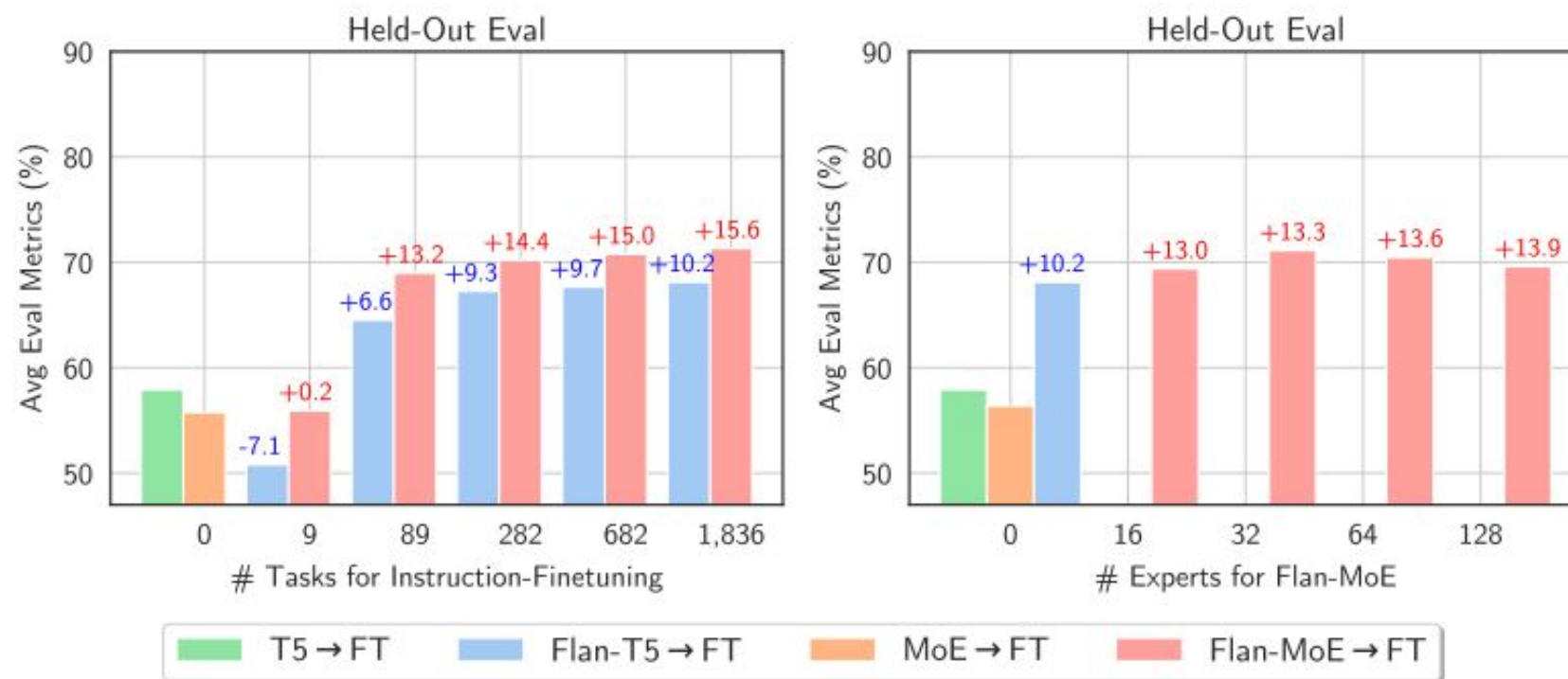
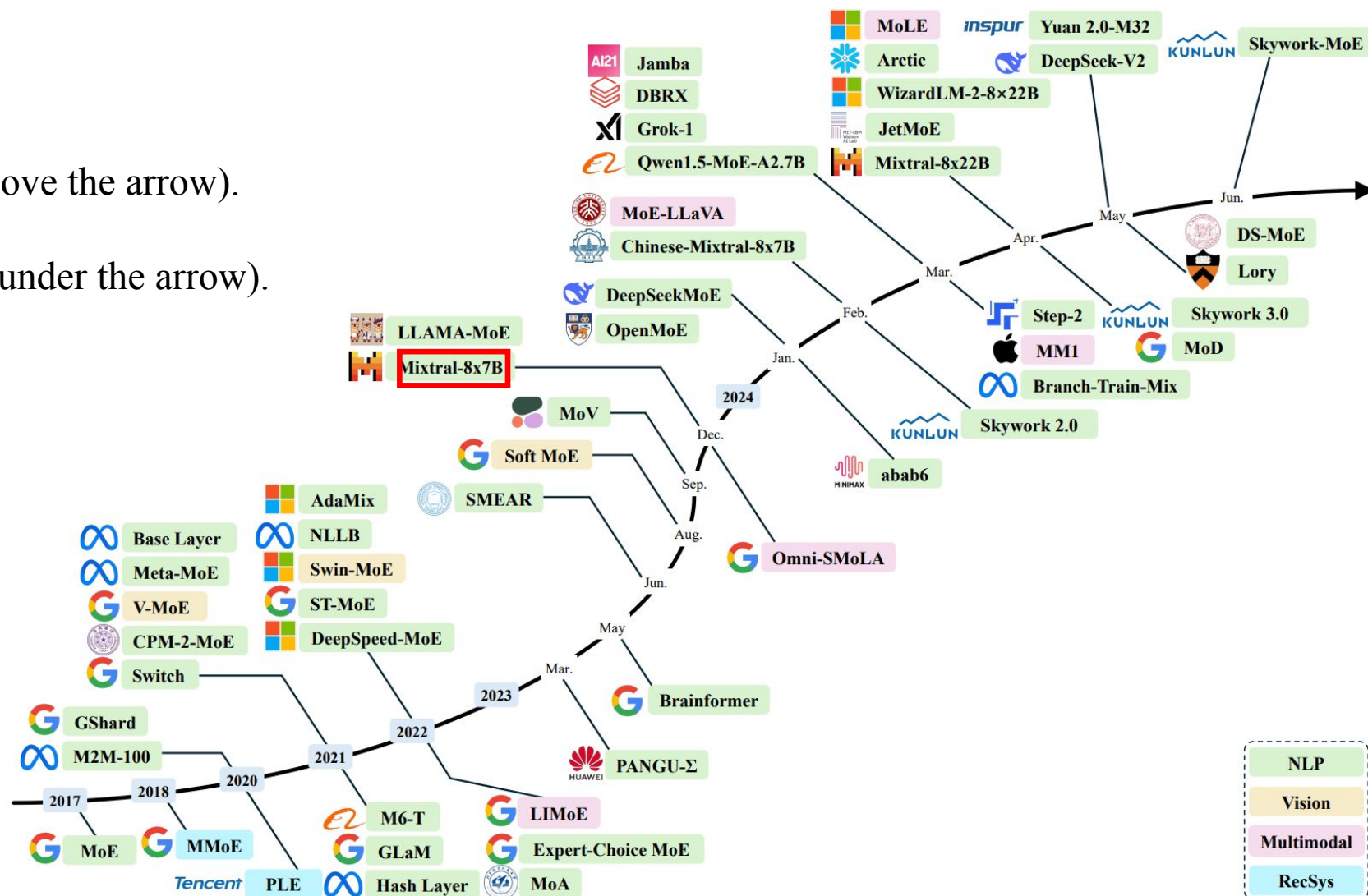


Figure 1: The effect of instruction tuning on MoE models versus dense counterparts for base-size models (same flops across all models in this figure). We perform single-task finetuning for each model on held-out benchmarks. **Compared to dense models, MoE models benefit more from instruction-tuning, and are more sensitive to the number of instruction-tuning tasks.** Overall, the performance of MoE models scales better with respect to the number of tasks, than the number of experts.

Many MoEs

■ Road Map

- Open-source (above the arrow).
- Private models (under the arrow).



MoE Design

What should we care when designing a MoE?

Network types	FFN, Attention
Fine-grained experts	64 experts/128 experts/...
Shared experts	Isolated experts
Activation Function	ReLU/GEGLU/SwiGLU
MoE frequency	Every two layer/Each layer/...
Training auxiliary loss	Auxiliary loss/Z-loss/...

Fine-grained and Shared Experts

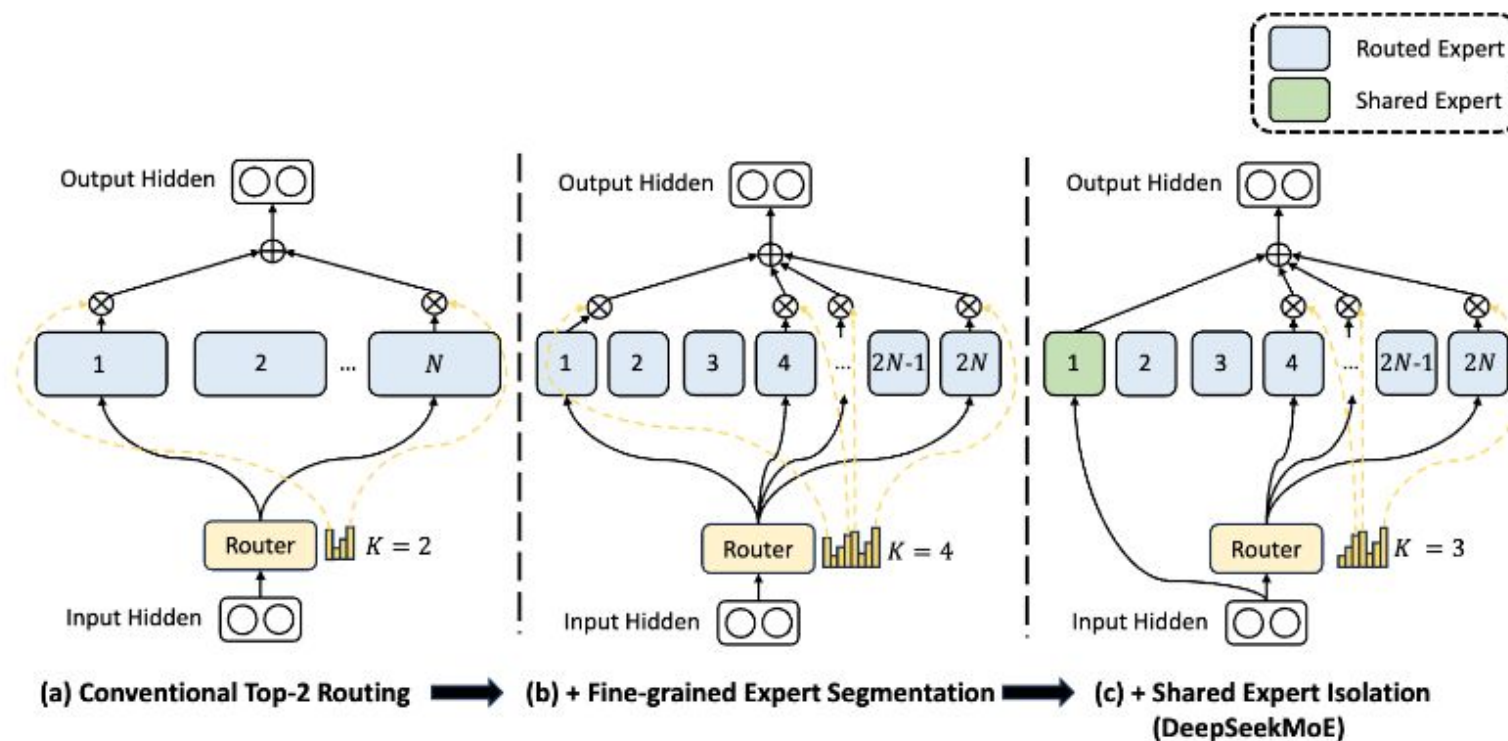


Figure 2 | Illustration of DeepSeekMoE. Subfigure (a) showcases an MoE layer with the conventional top-2 routing strategy. Subfigure (b) illustrates the fine-grained expert segmentation strategy. Subsequently, subfigure (c) demonstrates the integration of the shared expert isolation strategy, constituting the complete DeepSeekMoE architecture. It is noteworthy that across these three architectures, the number of expert parameters and computational costs remain constant.

MoE Experts Design

Reference	Models	Expert Count (Activ./Total)	d_{model}	d_{ffn}	d_{expert}	#L	#H	d_{head}	Placement Frequency	Activation Function	Share Expert Count
GShard [86] (2020)	600B	2/2048	1024	8192	d_{ffn}	36	16	128	1/2	ReLU	0
	200B	2/2048	1024	8192	d_{ffn}	12	16	128	1/2	ReLU	0
	150B	2/512	1024	8192	d_{ffn}	36	16	128	1/2	ReLU	0
	37B	2/128	1024	8192	d_{ffn}	36	16	128	1/2	ReLU	0
Switch [49] (2021)	7B	1/128	768	2048	d_{ffn}	12	12	64	1/2	GEGLU	0
	26B	1/128	1024	2816	d_{ffn}	24	16	64	1/2	GEGLU	0
	395B	1/64	4096	10240	d_{ffn}	24	64	64	1/2	GEGLU	0
	1571B	1/2048	2080	6144	d_{ffn}	15	32	64	1	ReLU	0
GLaM [44] (2021)	0.1B/1.9B	2/64	768	3072	d_{ffn}	12	12	64	1/2	GEGLU	0
	1.7B/27B	2/64	2048	8192	d_{ffn}	24	16	128	1/2	GEGLU	0
	8B/143B	2/64	4096	16384	d_{ffn}	32	32	128	1/2	GEGLU	0
	64B/1.2T	2/64	8192	32768	d_{ffn}	64	128	128	1/2	GEGLU	0
DeepSpeed-MoE [121] (2022)	350M/13B	2/128	1024	$4d_{model}$	d_{ffn}	24	16	64	1/2	GeLU	0
	1.3B/52B	2/128	2048	$4d_{model}$	d_{ffn}	24	16	128	1/2	GeLU	0
	PR-350M/4B	2/32-2/64	1024	$4d_{model}$	d_{ffn}	24	16	64	1/2, 10L-32E, 2L-64E	GeLU	1
	PR-1.3B/31B	2/64-2/128	2048	$4d_{model}$	d_{ffn}	24	16	128	1/2, 10L-64E, 2L-128E	GeLU	1
ST-MoE [197] (2022)	0.8B/4.1B	2/32	1024	2816	d_{ffn}	27	16	64	1/4, add extra FFN	GEGLU	0
	32B/269B	2/64	5120	20480	d_{ffn}	27	64	128	1/4, add extra FFN	GEGLU	0
Mixtral [74] (2023)	13B/47B	2/8	4096	14336	d_{ffn}	32	32	128	1	SwiGLU	0
	39B/141B	2/8	6144	16384	d_{ffn}	56	48	128	1	SwiGLU	0
LLAMA-MoE [149] (2023)	3.0B/6.7B	2/16	4096	11008	688	32	32	128	1	SwiGLU	0
	3.5B/6.7B	4/16	4096	11008	688	32	32	128	1	SwiGLU	0
	3.5B/6.7B	2/8	4096	11008	1376	32	32	128	1	SwiGLU	0
DeepSeekMoE [30] (2024)	0.24B/1.89B	8/64	1280	-	$\frac{1}{4}d_{ffn}$	9	10	128	1	SwiGLU	1
	2.8B/16.4B	8/66	2048	10944	1408	28	16	128	1, except 1st layer	SwiGLU	2
	22B/145B	16/132	4096	-	$\frac{1}{8}d_{ffn}$	62	32	128	1, except 1st layer	SwiGLU	4
OpenMoE [172] (2024)	339M/650M	2/16	768	3072	d_{ffn}	12	12	64	1/4	SwiGLU	1
	2.6B/8.7B	2/32	2048	8192	d_{ffn}	24	24	128	1/6	SwiGLU	1
	6.8B/34B	2/32	3072	12288	d_{ffn}	32	24	128	1/4	SwiGLU	1
Qwen1.5-MoE [151] (2024)	2.7B/14.3B	8/64	2048	5632	1408	24	16	128	1	SwiGLU	4
DBRX [34] (2024)	36B/132B	4/16	6144	10752	d_{ffn}	40	48	128	1	SwiGLU	0
Jamba [94] (2024)	12B/52B	2/16	4096	14336	d_{ffn}	32	32	128	1/2, 1:7 Attention:Mamba	SwiGLU	0
Skywork-MoE [154] (2024)	22B/146B	2/16	4608	12288	d_{ffn}	52	36	128	1	SwiGLU	0
Yuan 2.0-M32 [166] (2024)	3.7B/40B	2/32	2048	8192	d_{ffn}	24	16	256	1	SwiGLU	0

Key points:

- Most recent models place MoE each layer.
- Some of recent models apply Shared experts.

Pyramid Design of Experts



- Utilizes more experts in the last few layers as compared to previous layers
- Positive results compared with the baseline MoE

Model (num. params)	LAMBADA	PIQA	BoolQ	RACE-h	TriviaQA	WebQs
350M+MoE-128 (13B)	62.70	74.59	60.46	35.60	16.58	5.17
350M+PR-MoE-32/64 (4B)	63.65	73.99	59.88	35.69	16.30	4.73
1.3B+MoE-128 (52B)	69.84	76.71	64.92	38.09	31.29	7.19
1.3B+PR-MoE-64/128 (31B)	70.60	77.75	67.16	38.09	28.86	7.73

Auxiliary Loss

Training with different auxiliary loss:

Reference	Auxiliary Loss	Coefficient
Shazeer et al.[135], V-MoE[128]	$L_{importance} + L_{load}$	$w_{importance} = 0.1, w_{load} = 0.1$
GShard[86], Switch-T[49], GLaM[44], Mixtral-8x7B[74], DBRX[34], Jamba[94], DeepSeekMoE[30], DeepSeek-V2[36], Skywork-MoE[154]	L_{aux}	$w_{aux} = 0.01$
ST-MoE[197], OpenMoE[172], MoA[182], JetMoE [139]	$L_{aux} + L_z$	$w_{aux} = 0.01, w_z = 0.001$
Mod-Squad[21], Moduleformer[140], DS-MoE[117]	L_{MI}	$w_{MI} = 0.001$

Importance loss: encourages all experts to have equal importance

Load loss: ensure balanced loads

Auxiliary loss: mitigating load balance losses

Z-loss: improving training stability by penalizing large logits

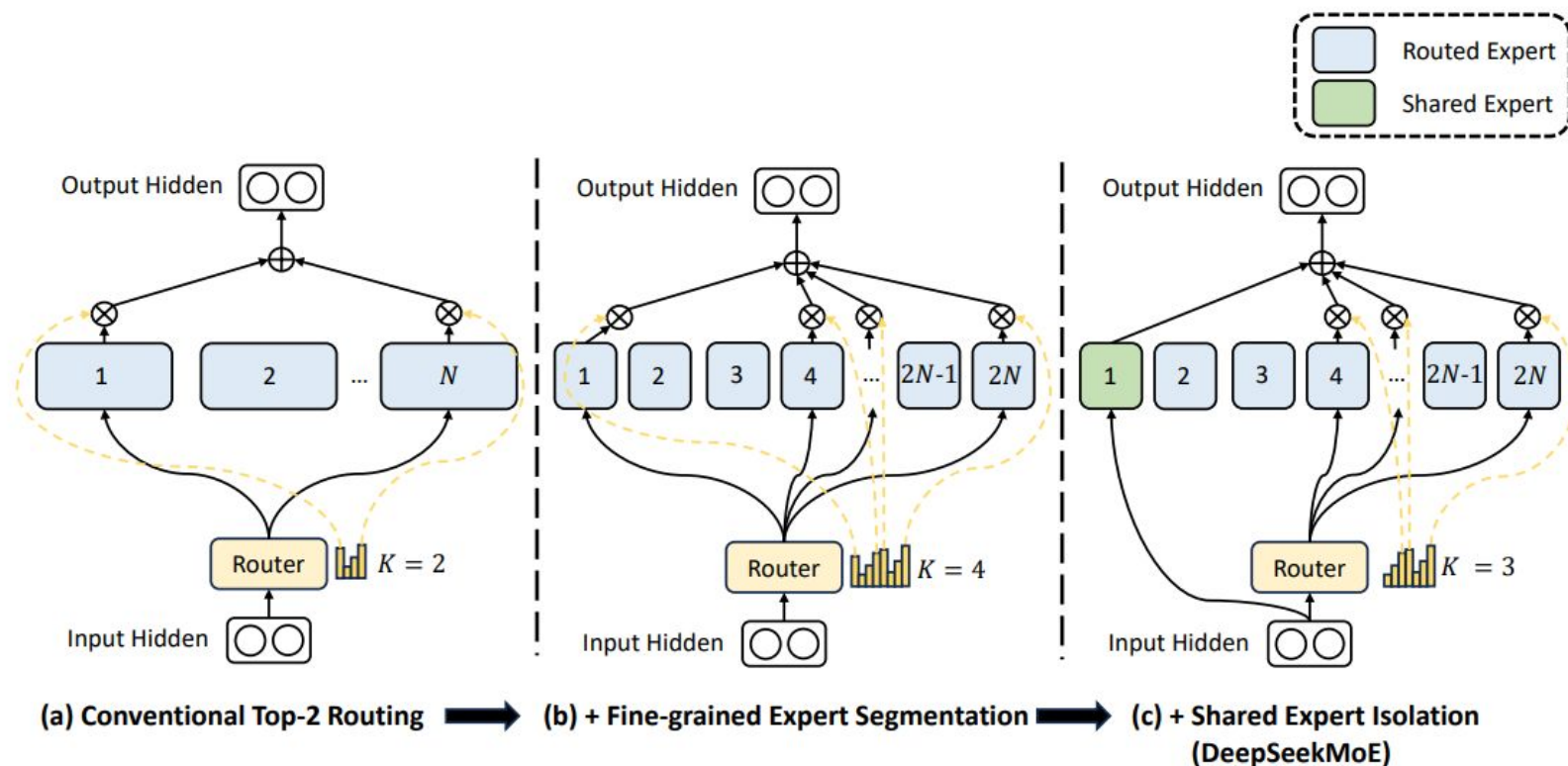
MI-loss: mutual information (MI) between experts and tasks to build task-expert alignment

Training MoE - Deepseek

Example 1: Deepseek-MoE

Deepseek-MoE 16B, total 16.4B parameters, 2.8B activate parameters.

Each MoE layer consists of 2 shared experts and 64 routed experts (select 6 experts).

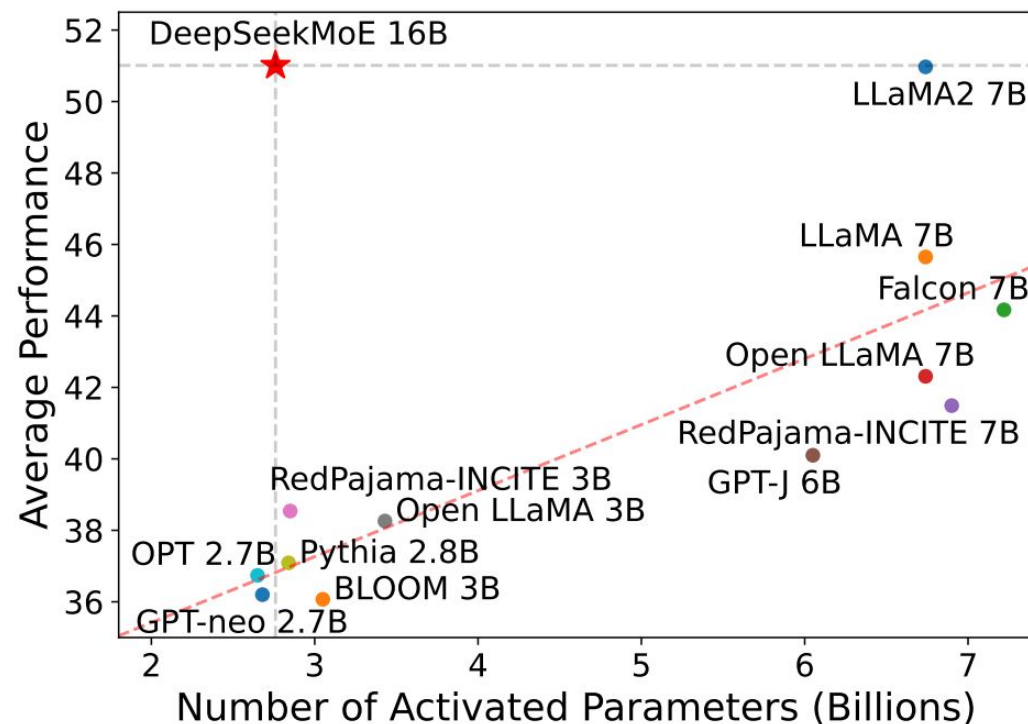


Key points:

- Fine-grained experts
- Shared experts

Training MoE - Deepseek

Example 1 : Deepseek-MoE

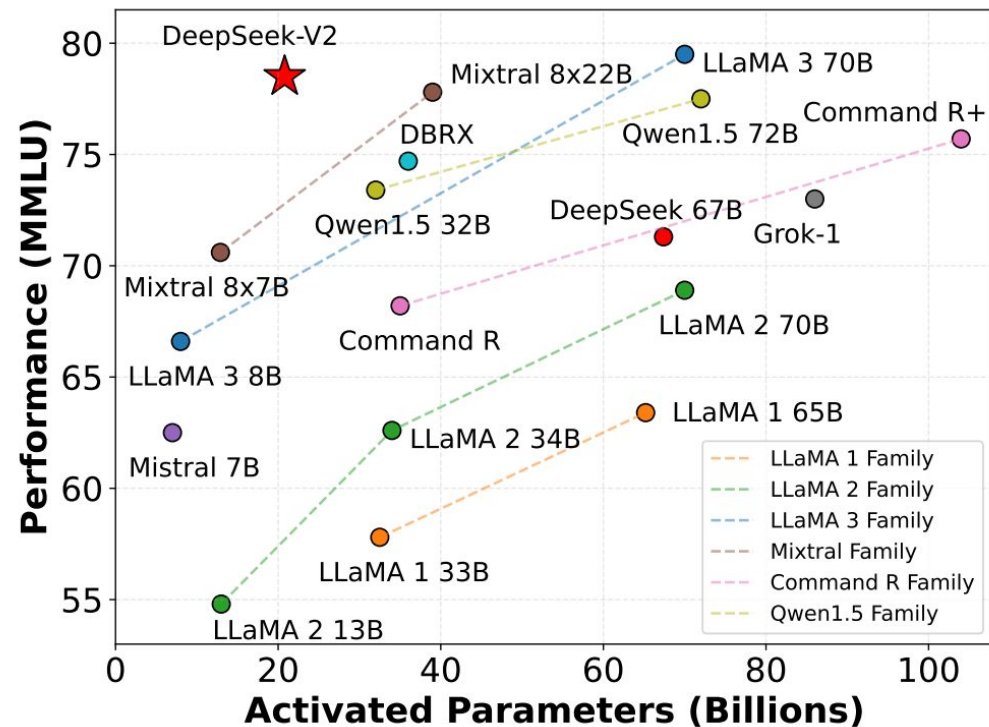


Metric	# Shot	DeepSeek 7B (Dense)	DeepSeekMoE 16B
# Total Params	N/A	6.9B	16.4B
# Activated Params	N/A	6.9B	2.8B
FLOPs per 4K Tokens	N/A	183.5T	74.4T
# Training Tokens	N/A	2T	2T
Pile (BPB)	N/A	0.75	0.74
HellaSwag (Acc.)	0-shot	75.4	77.1
PIQA (Acc.)	0-shot	79.2	80.2
ARC-easy (Acc.)	0-shot	67.9	68.1
ARC-challenge (Acc.)	0-shot	48.1	49.8
RACE-middle (Acc.)	5-shot	63.2	61.9
RACE-high (Acc.)	5-shot	46.5	46.4
DROP (EM)	1-shot	34.9	32.9
GSM8K (EM)	8-shot	17.4	18.8
MATH (EM)	4-shot	3.3	4.3
HumanEval (Pass@1)	0-shot	26.2	26.8
MBPP (Pass@1)	3-shot	39.0	39.2
TriviaQA (EM)	5-shot	59.7	64.8
NaturalQuestions (EM)	5-shot	22.2	25.5
MMLU (Acc.)	5-shot	48.2	45.0
WinoGrande (Acc.)	0-shot	70.5	70.2
CLUEWSC (EM)	5-shot	73.1	72.1
CEval (Acc.)	5-shot	45.0	40.6
CMMLU (Acc.)	5-shot	47.2	42.5
CHID (Acc.)	0-shot	89.3	89.4

Training MoE - Deepseek

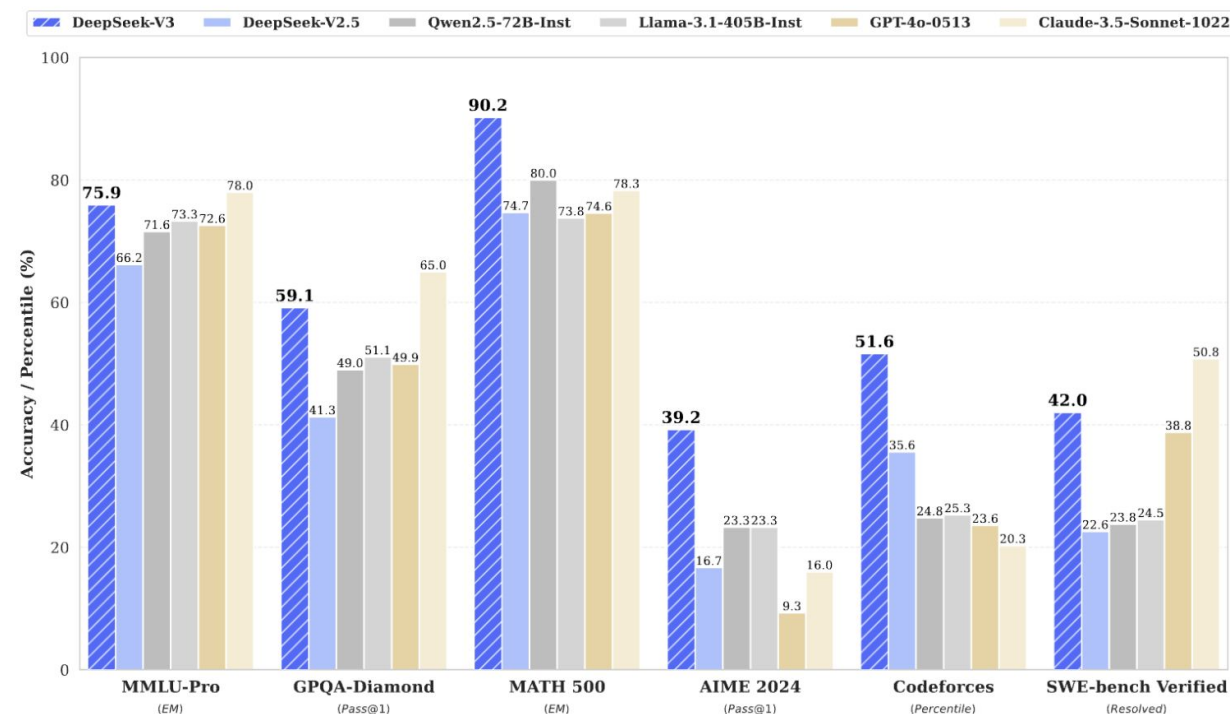
Deepseek-V2

236B total parameters, 21B are activated.
2 shared experts and 160 routed experts (6 select).



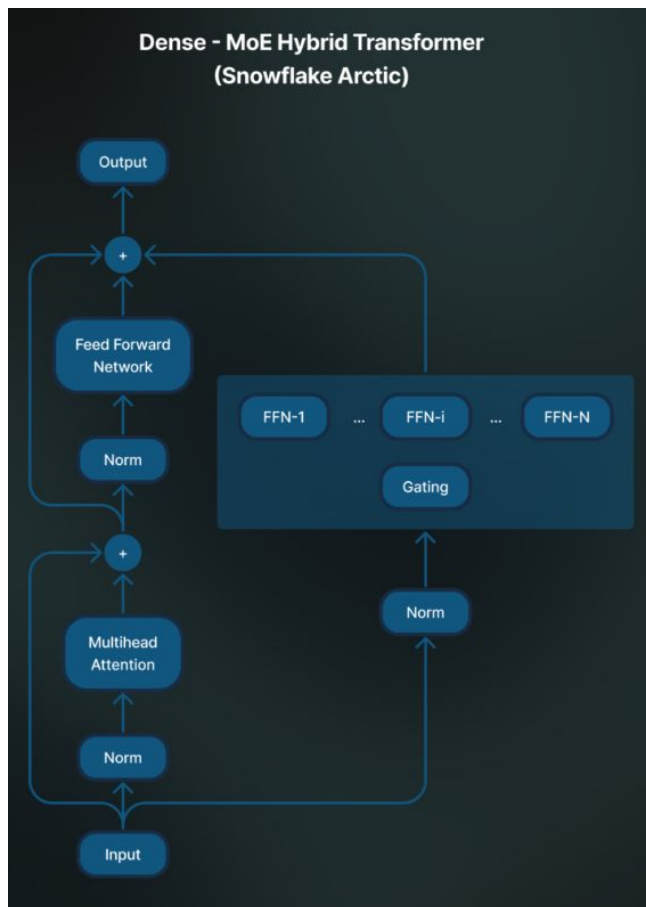
Deepseek-V3

MoE language model with 671B total parameters with 37B activated for each token



Training MoE - Arctic

Example 2: Arctic (Dense and Sparse)



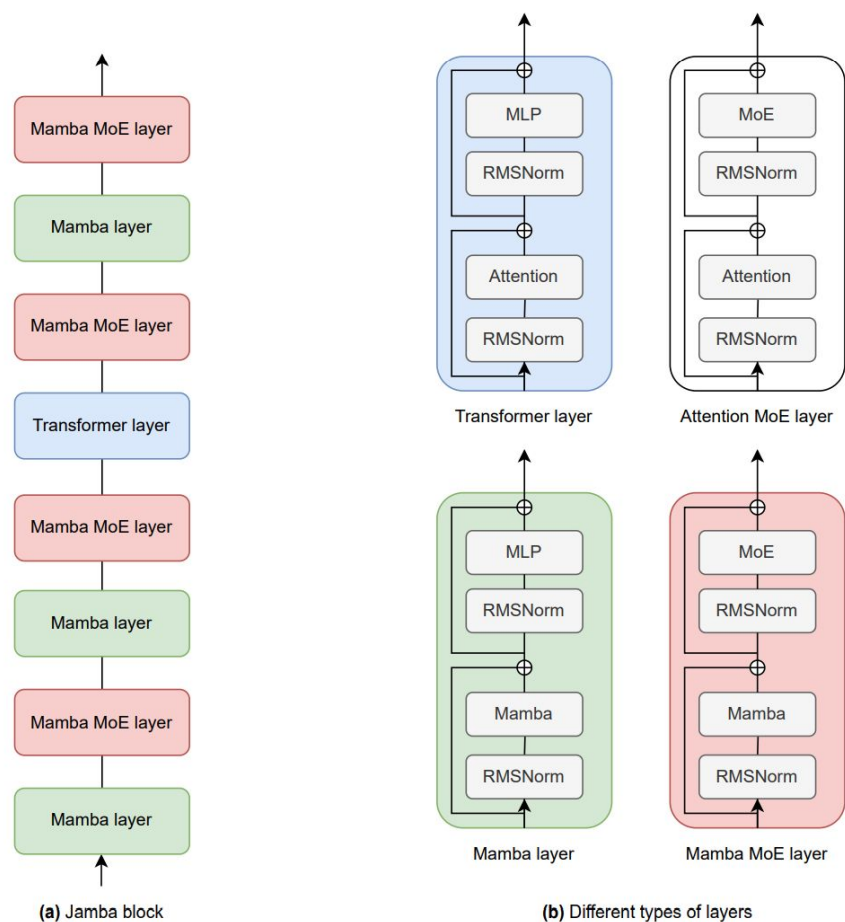
Arctic uses a unique Dense-MoE Hybrid transformer architecture.

- It combines a 10B dense transformer model with a residual $128 \times 3.66\text{B}$ MoE MLP.
- 480B total and 17B active parameters chosen using a top-2 gating.

	Snowflake Arctic	DBRX	Llama 3 8B	Llama 2 70B	Llama 3 70B	Mixtral 8x7B	Mixtral 8x22B
Active Parameters	17B	36B	8B	70B	70B	13B	44B
ENTERPRISE							
SQL Generation (Spider)	79.0	76.3	69.9	62.8	80.2	71.3	79.2
Coding (HumanEval+, MBPP+)	64.3	61.0	59.2	33.7	71.9	48.1	69.9
Instruction Following (IFEval)	57.4	54.8	42.7	-	43.6	52.2	61.5
ACADEMIC							
Math (GSM8K)	74.2	73.5	75.4	52.6	91.4	63.2	84.2
Common Sense (Avg of 11 metrics)	73.1	74.8	68.5	72.1	72.6	74.1	75.6
World Knowledge (MMLU)	67.3	73.3	65.7	68.6	79.8	70.4	77.5

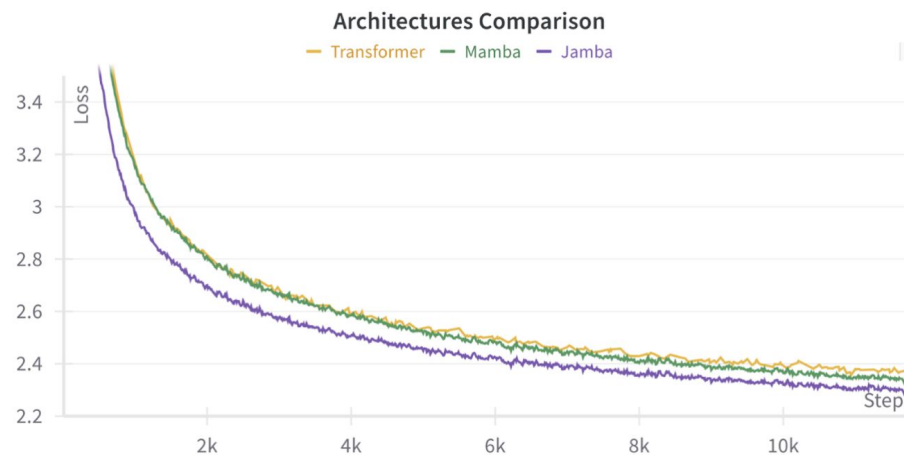
Training MoE - Jamba

Example 3: Jamba (Hybrid architecture)



Jamba is a hybrid decoder architecture that mixes Transformer layers with Mamba layers, in addition to a mixture-of-experts (MoE) module.

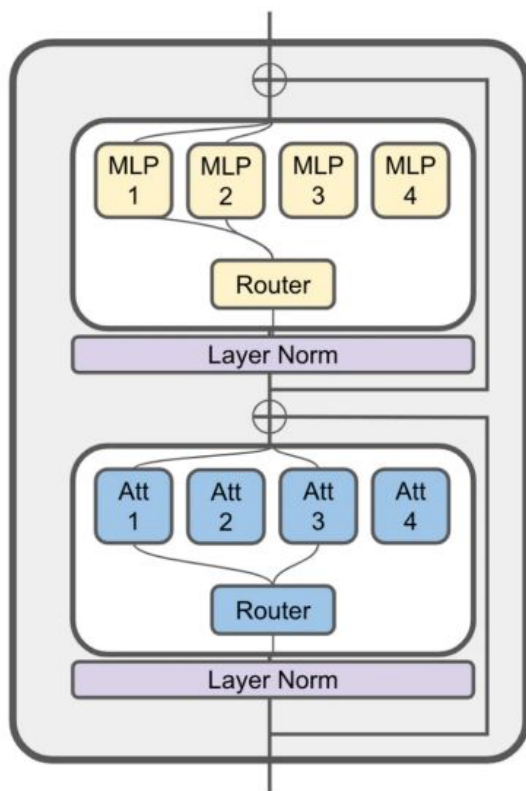
	Available params	Active params
LLAMA-2	6.7B	6.7B
Mistral	7.2B	7.2B
Mixtral	46.7B	12.9B
Jamba	52B	12B



Jamba: A Hybrid Transformer-Mamba Language Model

Training MoE - JetMoE

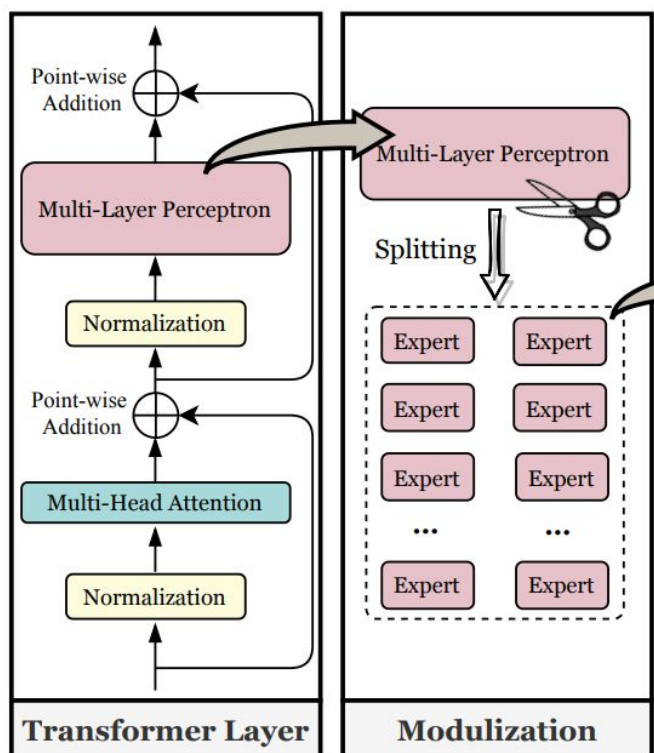
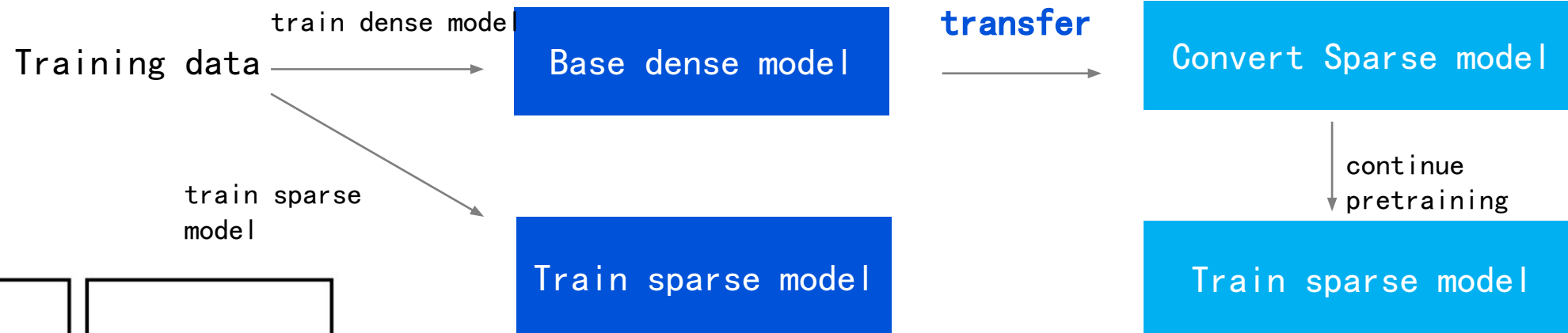
Example 4: JetMoE



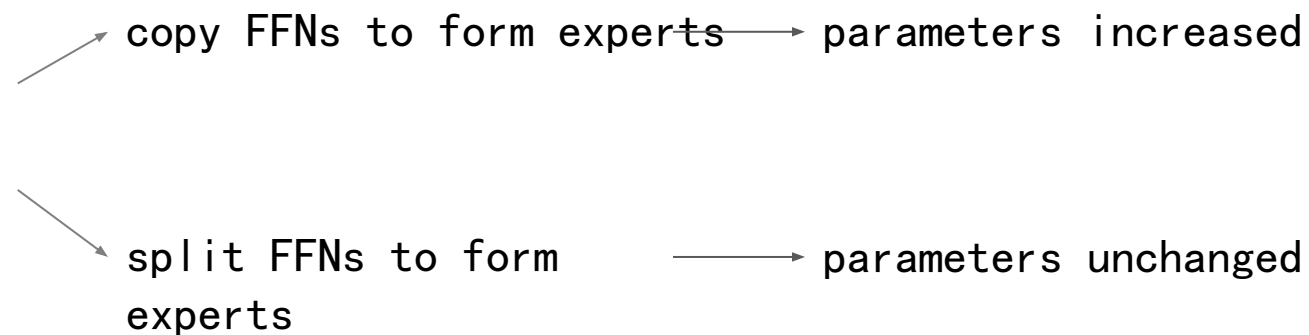
Both attention and feedforward layers are sparsely activated, allowing JetMoE-8B to have 8B parameters while only activating 2B for each input token.

	LLaMA2	DeepseekMoE	Gemma	JetMoE
# Total Params	7B	16B	2B	8B
# Activate Params	7B	2.8B	2B	2.2B
# Training tokens	2T	2T	2T	1.25T
ARC-challenge	53.1	53.2	48.4	48.7
Hellaswag	78.6	79.8	71.8	80.5
MMLU	46.9	46.3	41.8	49.2
TruthfulQA	38.8	36.1	33.1	41.7
WinoGrande	74.0	73.7	66.3	70.2
GSM8k	14.5	17.3	16.9	27.8
OpenLLM Leaderboard Avg.	51.0	51.1	46.4	53.0
MBPP (Pass@1)	20.8	34.0	28.0	34.2
HumanEval (Pass@1)	12.8	25.0	24.4	14.6
All Avg.	45.5	47.3	43.2	47.6

Building MoE LLMs



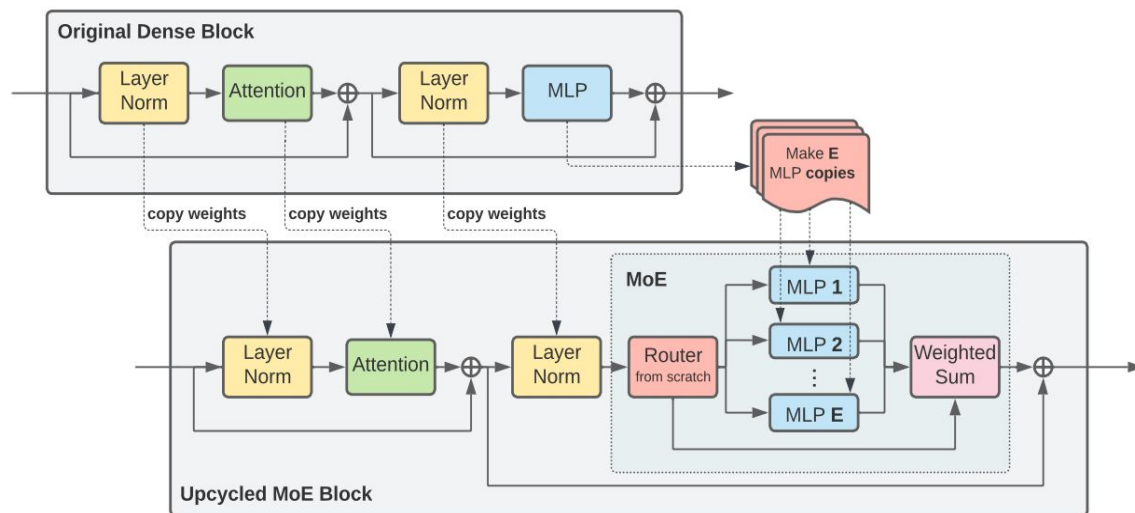
Sparse transformation



Building MoE from Dense LLMs

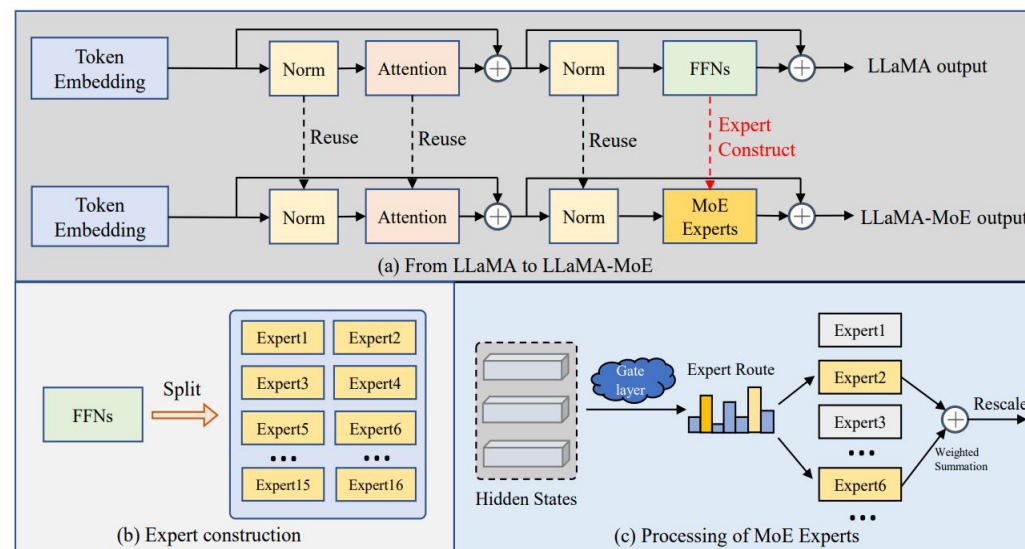
Two routes to build MoE from Dense models:

- **Sparse Upcycling**



Copying the FFNs to form experts

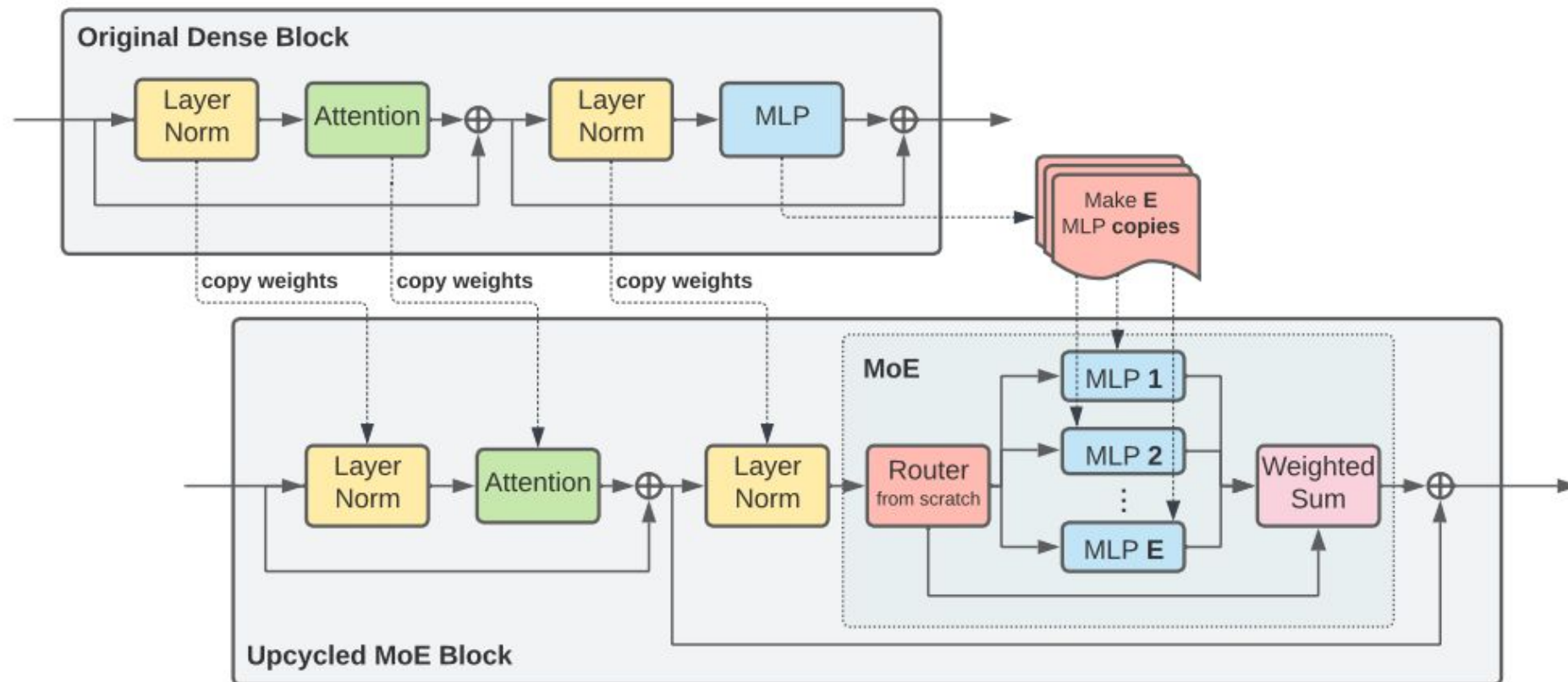
- **Sparse Splitting (MoEfication)**



Splitting the FFNs to form experts

Sparse Upcycling

Sparse upcycling solution



copying the MLP layers:

Upcycled T5-Large and T5-Base models outperform their dense counterparts by 1.5–2 absolute points on SuperGLUE using 46% and 55% extra training, respectively.

Sparse Upcycling - Mixtral MoE

Upcycling from Dense to MoE?

Example 1 : Mixtral 8×22B(7B) (April, 2024)

Total 141B parameters, 39B activate parameters, (8 experts and 2 experts are selected)

		Common sense and reasoning					Knowledge		
Model	Active parameters	MMLU	HellaS	WinoG	Arc C (5)	Arc C (25)	TriQA	NaturalQS	
CC-BY-NC license	LLaMA 2 70B	70B	69.9%	87.1%	83.2%	86.0%	85.1%	77.57%	35.5%
	Command R	35B	68.2%	87.0%	81.5%	-	66.5%	-	-
	Command R+	104B	75.7%	88.6%	85.4%	-	71.0%	-	-
	Mistral 7B	7B	62.47%	83.1%	78.0%	77.2%	78.1%	68.8%	28.1%
	Mixtral 8x7B	12.9B	70.63%	86.6%	81.2%	85.8%	85.9%	78.4%	36.5%
	Mixtral 8x22B	39B	77.75%	88.5%	84.7%	91.3%	91.3%	82.2%	40.1%

Sparse Upcycling - Mixtral MoE

Upcycling from Dense to MoE?

Hugging Face

mistralai/Mixtral-8x7B-v0.1

Text Generation Transformers Safetensors 5 languages mistral Inference Endpoints

text-generation-inference License: apache-2.0

Model card Files Community 15

Mixtral-8x7B-v0.1 / config.json

pstock Update config.json (#5) c56a162

```
1 {
2   "architectures": [
3     "MixtralForCausalLM"
4   ],
5   "attention_dropout": 0.0,
6   "bos_token_id": 1,
7   "eos_token_id": 2,
8   "hidden_act": "silu",
9   "hidden_size": 4096,
10  "initializer_range": 0.02,
11  "intermediate_size": 14336,
12  "max_position_embeddings": 32768,
13  "model_type": "mixtral",
14  "num_attention_heads": 32,
15  "num_experts_per_tok": 2,
16  "num_hidden_layers": 32,
17  "num_key_value_heads": 8,
18  "num_local_experts": 8,
19  "output_router_logits": false,
20  "rms_norm_eps": 1e-05,
21  "rope_theta": 1000000.0,
22  "router_aux_loss_coef": 0.02,
23  "sliding_window": 4096,
24  "tie_word_embeddings": false,
25  "torch_dtype": "bfloat16"
```

Hugging Face

mistralai/Mistral-7B-v0.1

Text Generation Transformers PyTorch Safetensors English mistral pretrained

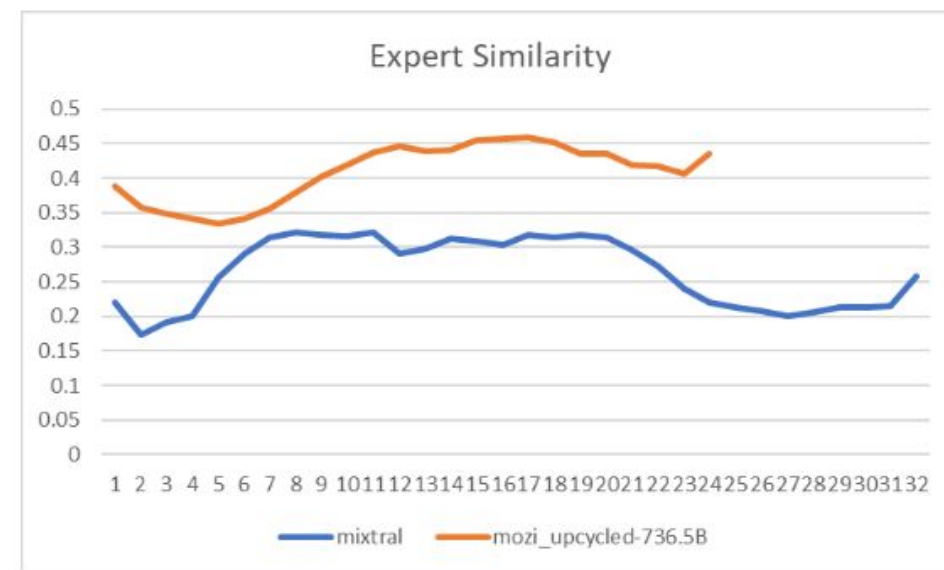
Inference Endpoints text-generation-inference arxiv:2310.06825 License: apache-2.0

Model card Files Community 97

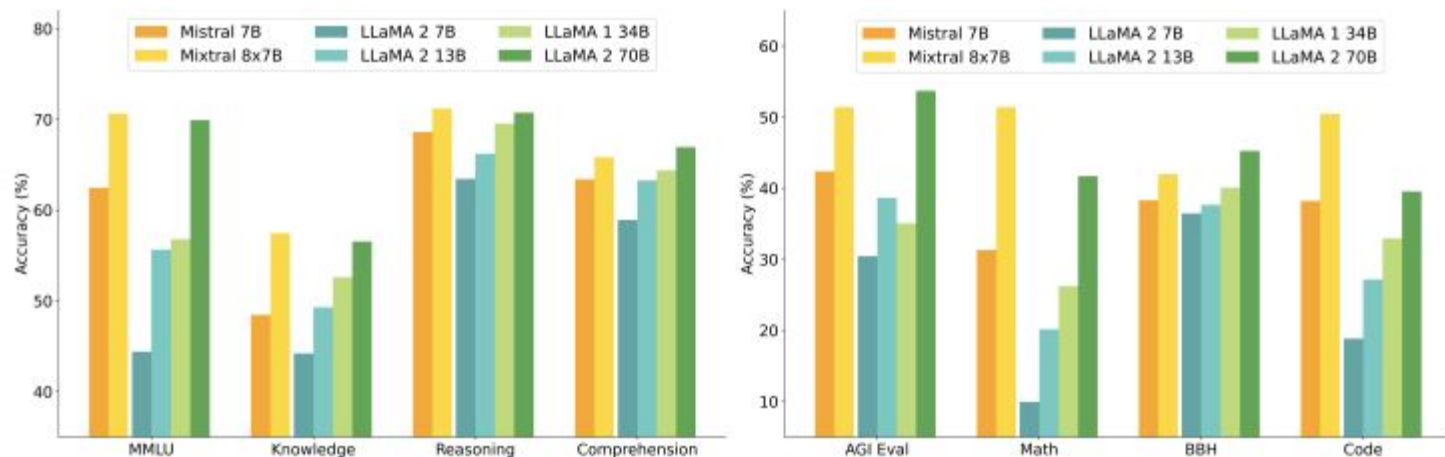
Mistral-7B-v0.1 / config.json

Bam4d Upload folder using huggingface_hub e836d8f

```
1 {
2   "architectures": [
3     "MixtralForCausalLM"
4   ],
5   "bos_token_id": 1,
6   "eos_token_id": 2,
7   "hidden_act": "silu",
8   "hidden_size": 4096,
9   "initializer_range": 0.02,
10  "intermediate_size": 14336,
11  "max_position_embeddings": 32768,
12  "model_type": "mistral",
13  "num_attention_heads": 32,
14  "num_hidden_layers": 32,
15  "num_key_value_heads": 8,
16  "rms_norm_eps": 1e-05,
17  "rope_theta": 10000.0,
18  "sliding_window": 4096,
19  "tie_word_embeddings": false,
20  "torch_dtype": "bfloat16",
21  "transformers_version": "4.34.0.dev0",
22  "use_cache": true,
23  "vocab_size": 32000
24 }
```



Mixtral 8x7B



Model	Active Params	MMLU	HellaS	WinoG	PIQA	Arc-e	Arc-c	NQ	TriQA	HumanE	MBPP	Math	GSM8K
LLaMA 2 7B	7B	44.4%	77.1%	69.5%	77.9%	68.7%	43.2%	17.5%	56.6%	11.6%	26.1%	3.9%	16.0%
LLaMA 2 13B	13B	55.6%	80.7%	72.9%	80.8%	75.2%	48.8%	16.7%	64.0%	18.9%	35.4%	6.0%	34.3%
LLaMA 1 33B	33B	56.8%	83.7%	76.2%	82.2%	79.6%	54.4%	24.1%	68.5%	25.0%	40.9%	8.4%	44.1%
LLaMA 2 70B	70B	69.9%	85.4%	80.4%	82.6%	79.9%	56.5%	25.4%	73.0%	29.3%	49.8%	13.8%	69.6%
Mistral 7B	7B	62.5%	81.0%	74.2%	82.2%	80.5%	54.9%	23.2%	62.5%	26.2%	50.2%	12.7%	50.0%
Mixtral 8x7B	13B	70.6%	84.4%	77.2%	83.6%	83.1%	59.7%	30.6%	71.5%	40.2%	60.7%	28.4%	74.4%

Table 2: Comparison of Mixtral with Llama. Mixtral outperforms or matches Llama 2 70B performance on almost all popular benchmarks while using 5x fewer active parameters during inference.

Mixtral 8x7B

Model ▲	🏆 Arena Elo rating ▲	📄 MT-bench (score) ▲	License ▲
GPT-4-Turbo	1243	9.32	Proprietary
GPT-4-0314	1192	8.96	Proprietary
GPT-4-0613	1158	9.18	Proprietary
Claude-1	1149	7.9	Proprietary
Claude-2.0	1131	8.06	Proprietary
Mixtral-8x7b-Instruct-v0.1	1121	8.3	Apache 2.0
Claude-2.1	1117	8.18	Proprietary
GPT-3.5-Turbo-0613	1117	8.39	Proprietary
Gemini Pro	1111		Proprietary
Claude-Instant-1	1110	7.85	Proprietary
Tulu-2-DPO-70B	1110	7.89	AI2 ImpACT Low-risk
Yi-34B-Chat	1110		Yi License
GPT-3.5-Turbo-0314	1105	7.94	Proprietary
Llama-2-70b-chat	1077	6.86	Llama 2 Community

Figure 6: LMSys Leaderboard. (Screenshot from Dec 22, 2023) Mixtral 8x7B Instruct v0.1 achieves an Arena Elo rating of 1121 outperforming Claude-2.1 (1117), all versions of GPT-3.5-Turbo (1117 best), Gemini Pro (1111), and Llama-2-70b-chat (1077). Mixtral is currently the best open-weights model by a large margin.

Sparse Upcycling - Skywork-MoE

Example 2: Skywork-MoE (June, 2024)

Total 146B parameters, 22B activate parameters, (16 experts and 2 experts are selected)

Initialize from Skywork-13B

	#AP	#TP	CEVAL	CMMLU	MMLU	GSM8K	MATH	HumanEval
Deepseek-67B	67	67	66.1	70.8	71.3	63.4	18.7	42.7
Qwen1.5-72B	72	72	84.1	83.5	77.5	79.5	34.1	41.5
Llama2-70B	70	70	-	-	68.9	56.8	13.6	29.9
Llama3-70B	70	70	-	-	78.8	82.7	36.7	39.0
Mixtral 8*7B	13	47	-	-	70.6	58.4	28.4	40.2
Mixtral 8*22B	39	141	-	-	77.8	78.6	41.8	45.1
Grok-1	86	314	-	-	73.0	62.9	23.9	63.2
DBRX-Instruct	36	132	-	-	73.7	66.9	-	70.1
Deepseek-V2	21	236	81.7	84.0	78.5	79.2	43.6	48.8
Skywork-13B	13	13	62.1	62.4	62.7	60.2	8.4	18.9
Skywork-MoE	22	146	82.2	79.5	77.4	76.1	31.9	43.9

Sparse Upcycling - Qwen-MoE

Example 3: Qwen1.5-MoE-A2.7B (Mar, 2024)

Upcycled from Qwen-1.8B, 14.3B parameters in total and 2.7B activated parameters.

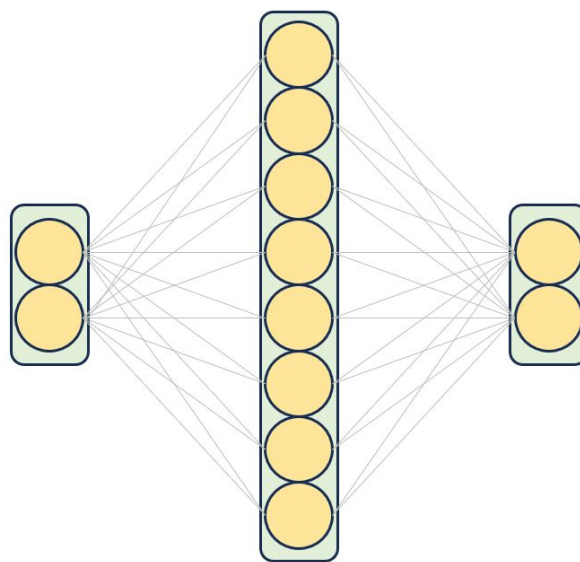
- Fine-grained experts (total 64 experts)
- use shared (4 experts) and routing experts (60 experts, choose 4)

Model	MMLU	GSM8K	HumanEval	Multilingual	MT-Bench
Mistral-7B	64.1	47.5	27.4	40.0	7.60
Gemma-7B	64.6	50.9	32.3	-	-
Qwen1.5-7B	61.0	62.5	36.0	45.2	7.60
DeepSeekMoE 16B	45.0	18.8	26.8	-	6.93
Qwen1.5-MoE-A2.7B	62.5	61.5	34.2	40.8	7.17

A remarkable reduction of 75%
in training

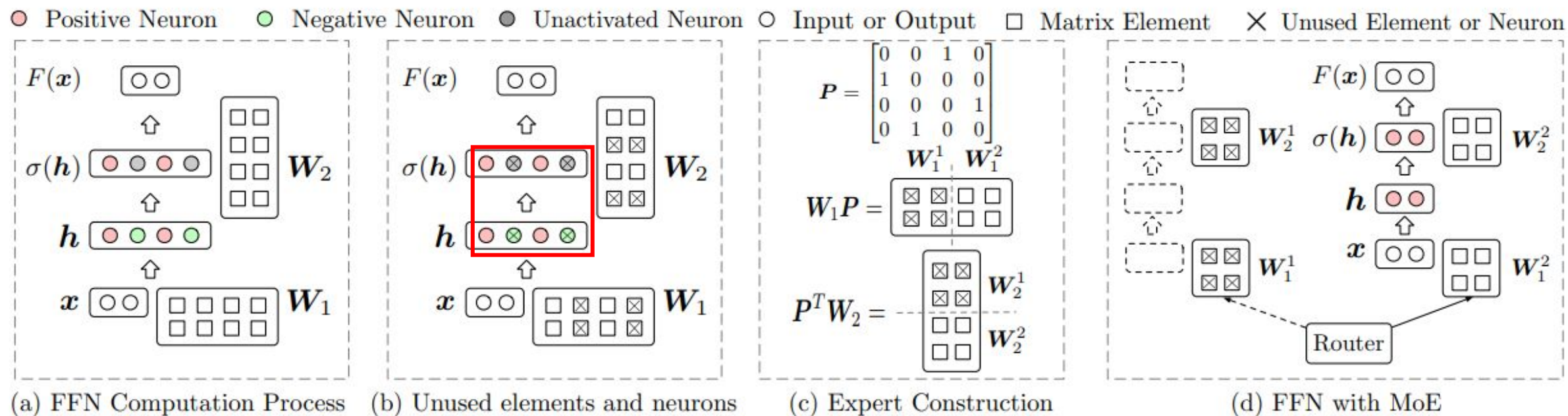
Sparse Splitting

Original FFN



Sparse Splitting - MoEfication

One solution for sparse splitting – MoEfication

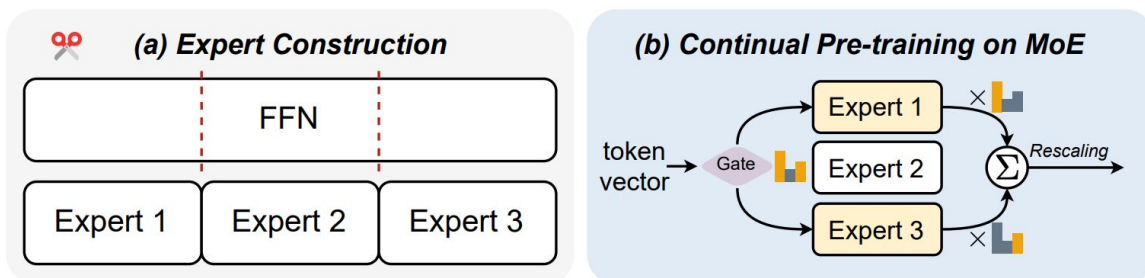


Splitting the FFN layers based on the activation diversity of different neurons.

Sparse Splitting - LLaMA-MoE

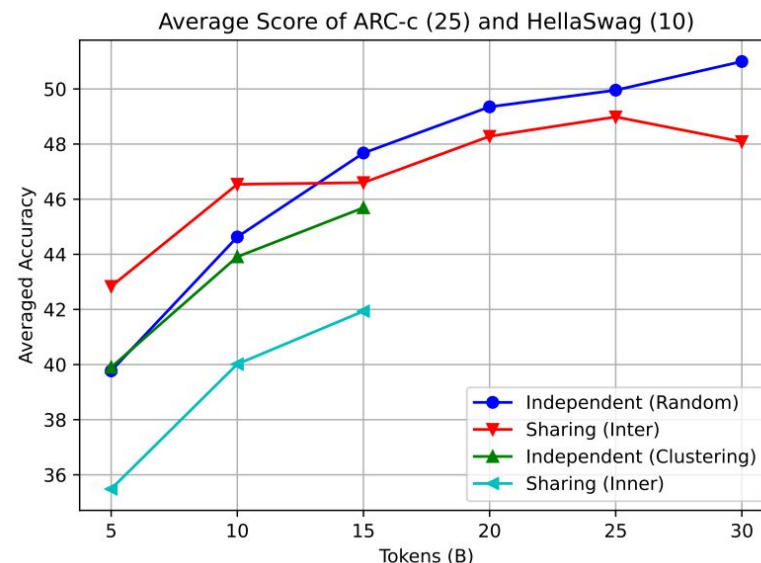
Sparsifying from Dense to MoE

Example 1: LLaMA-MoE (Dec, 2023)



Explore different FFN splitting strategies:

- Neuro-Independent
 1. Random splitting the FFNs
 2. Clustering with n centroids
- Neuro-Sharing
 1. Obtain n importance vectors
 2. Set aside the neuros shared by most experts and then obtain n importance vectors



Random splitting obtains the best.

LLaMA-MoE: Building Mixture-of-Experts from LLaMA with Continual Pre-training

Sparse Splitting - LLama-MoE

Sparsifying from Dense to MoE

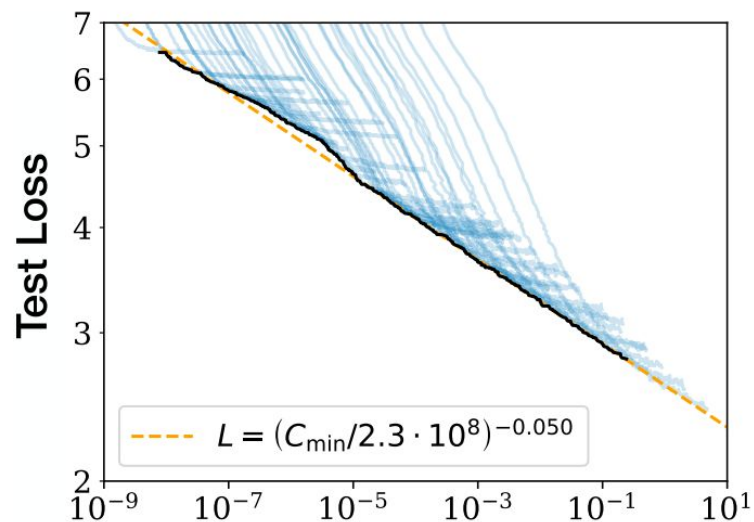
Model	Commonsense & Reading Comprehension					
	SciQ	PIQA	WinoGrande	ARC-E	ARC-C (25)	HellaSwag (10)
OPT-2.7B	78.9	74.8	60.8	54.4	34.0	61.4
Pythia-2.8B	83.2	73.6	59.6	58.8	36.7	60.7
INCITE-Base-3B	85.6	73.9	63.5	61.7	40.3	64.7
Open-LLaMA-3B-v2	88.0	77.9	63.1	63.3	40.1	71.4
Sheared-LLaMA-2.7B	87.5	76.9	65.0	63.3	41.6	71.0
LLaMA-MoE-3.0B	84.2	77.5	63.6	60.2	40.9	70.8
LLaMA-MoE-3.5B (4/16)	87.6	77.9	65.5	65.6	44.2	73.3
LLaMA-MoE-3.5B (2/8)	88.4	77.6	66.7	65.3	43.1	73.3

Model	Continued		LM	World Knowledge		Average
	LogiQA	BoolQ (32)	LAMBADA	NQ (32)	MMLU (5)	
OPT-2.7B	25.8	63.3	63.6	10.7	25.8	50.3
Pythia-2.8B	28.1	65.9	64.6	8.7	26.8	51.5
INCITE-Base-3B	27.5	65.8	65.4	15.2	27.2	53.7
Open-LLaMA-3B-v2	28.1	69.2	67.4	16.0	26.8	55.6
Sheared-LLaMA-2.7B	28.3	73.6	68.3	17.6	27.3	56.4
LLaMA-MoE-3.0B	30.6	71.9	66.6	17.0	26.8	55.5
LLaMA-MoE-3.5B (4/16)	29.7	75.0	69.5	20.3	26.8	57.7
LLaMA-MoE-3.5B (2/8)	29.6	73.9	69.4	19.8	27.0	57.6

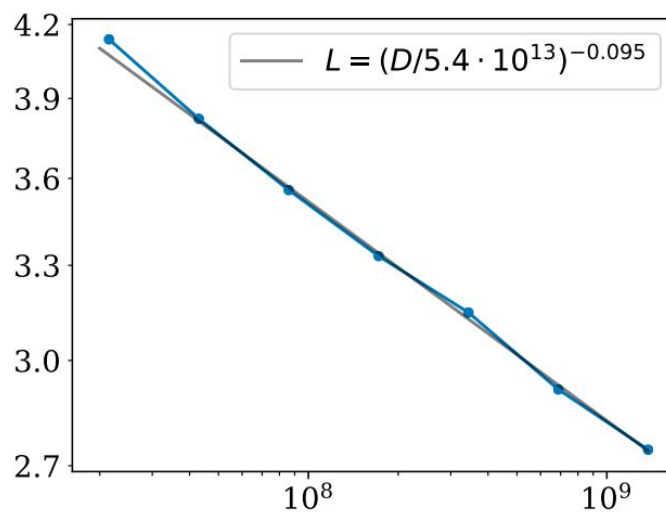
With 200B tokens continual pretraining,

LLaMA-MoE surpasses dense models with similar activation parameters.

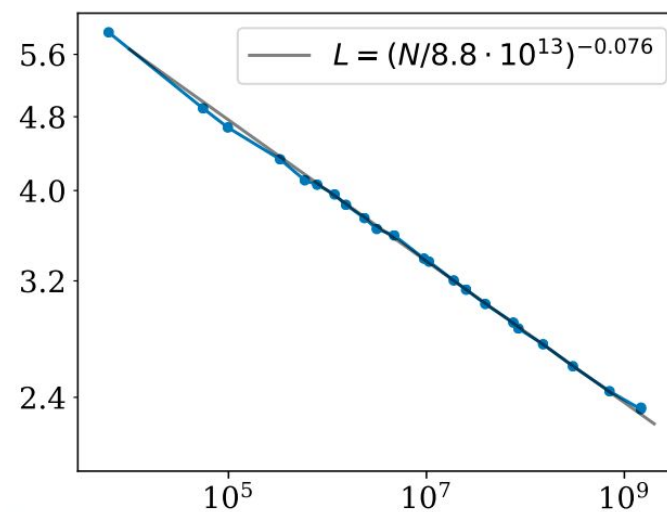
Scaling Law



Compute
PF-days, non-embedding



Dataset Size
tokens



Parameters
non-embedding

Upstream Scaling (Pre-training)

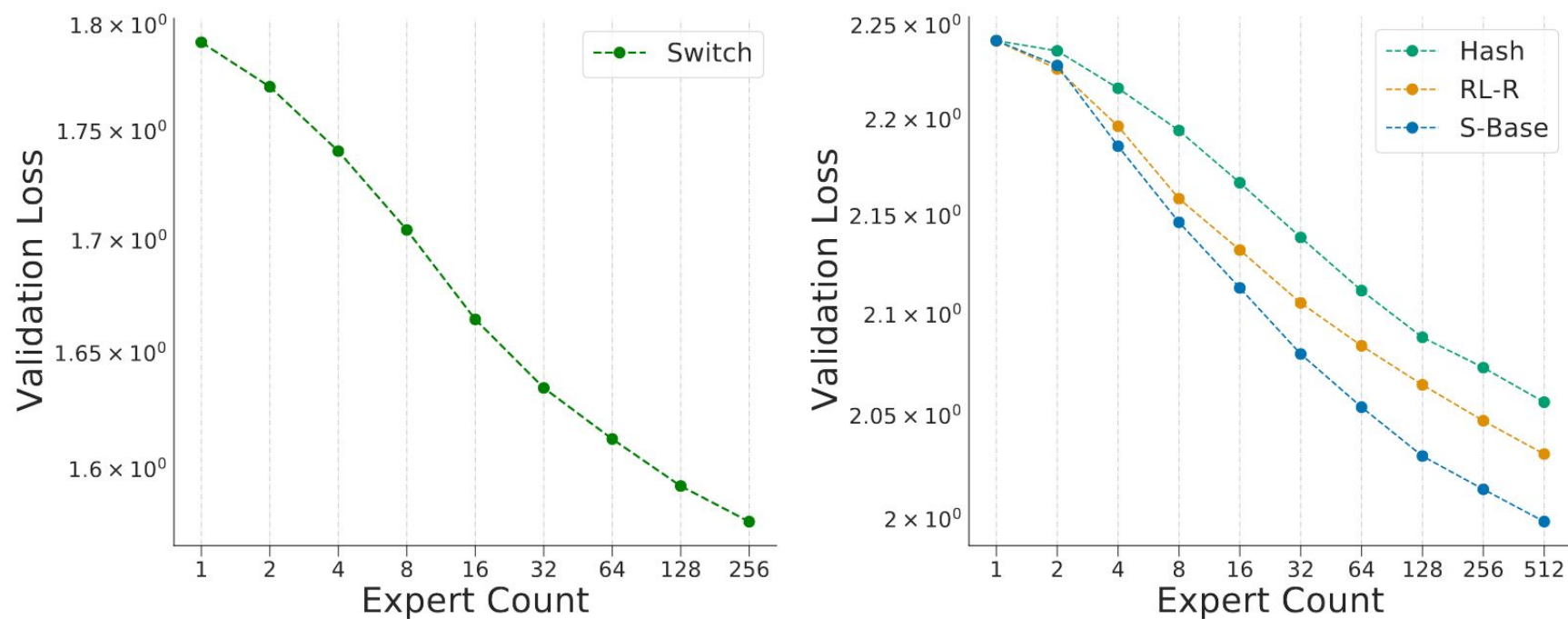


Figure 3: **Sparse scaling plots with expert count.** The cross-entropy scaling plots as a function of the number of experts are shown from [Fedus et al. \(2021\)](#) (**left**) and the three sparse variants from [Clark et al. \(2022\)](#), S-Base, RL-R, Hash (**right**). The top left-most point in both plots is an approximately compute-matched dense model. As the expert count increases, the models become increasingly sparse and yield lower validation losses.

Downstream Scaling (Fine-tuning)

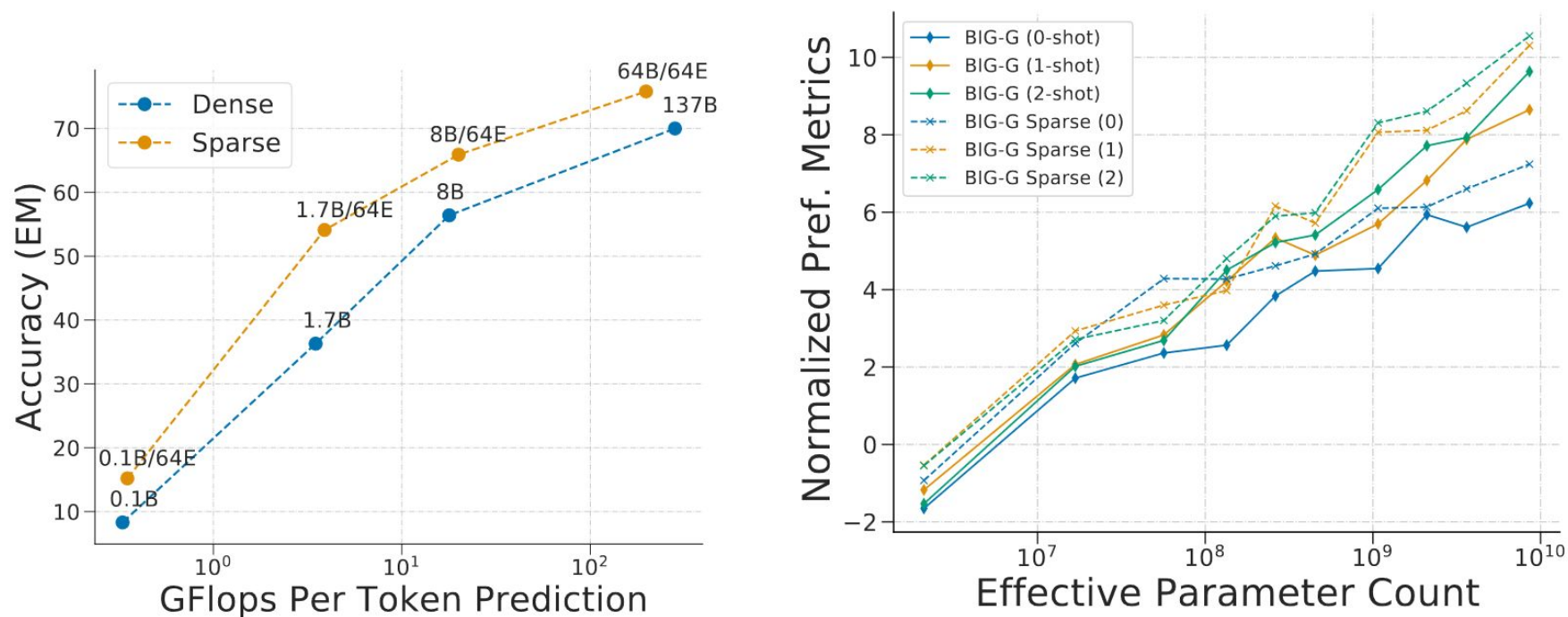


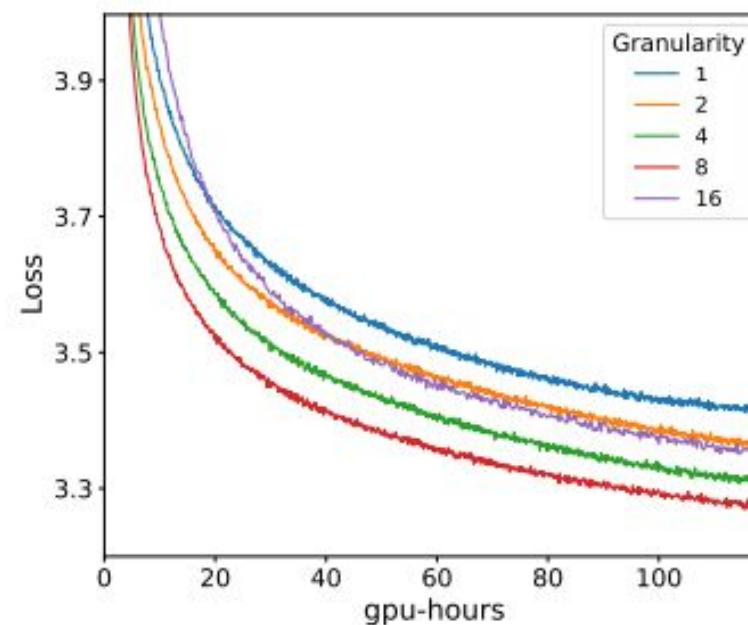
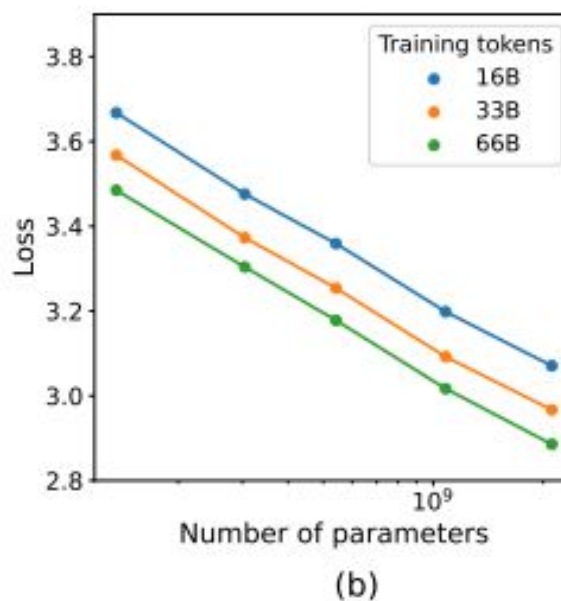
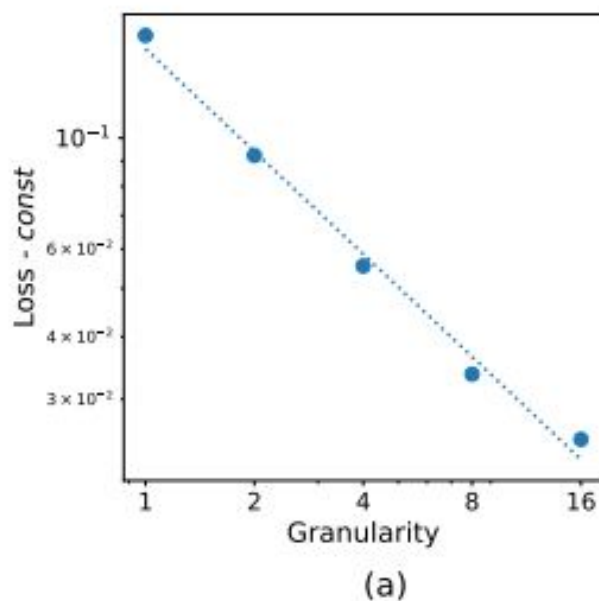
Figure 4: **Sparse scaling for few-shot inference.** **Left:** Du et al. (2021) measures the few-shot inference performance on TriviaQA, demonstrating consistent gains of sparse MoE models over dense models up to 137B parameters. Each label, such as 8B/64E, says how many parameters per input are used (8B) and how many experts (64E). **Right:** BigBench (Srivastava et al., 2022) studied the few-shot scaling properties on a larger set of 161 contributed JSON tasks to confirm improvements of sparse expert models over their FLOP-matched dense counterparts.

$$\bar{N} \triangleq (N)^{\alpha(\hat{E})/\alpha(E_{\text{start}})} \left(\hat{E}/E_{\text{start}} \right)^{b/\alpha(E_{\text{start}})}$$
$$\alpha(E) = a + c \log E$$

“The size $\bar{N}$ of a dense model giving the same performance as a Routing Network.”

Scaling (Fine-grained Experts)

- Granularity denotes the size as the feed-forward layer divides the inner dimension of each expert network;
- Increasing granularity results in a lower loss, in different numbers of training tokens;
- Considering GPU-hours, the conclusion is slightly different.

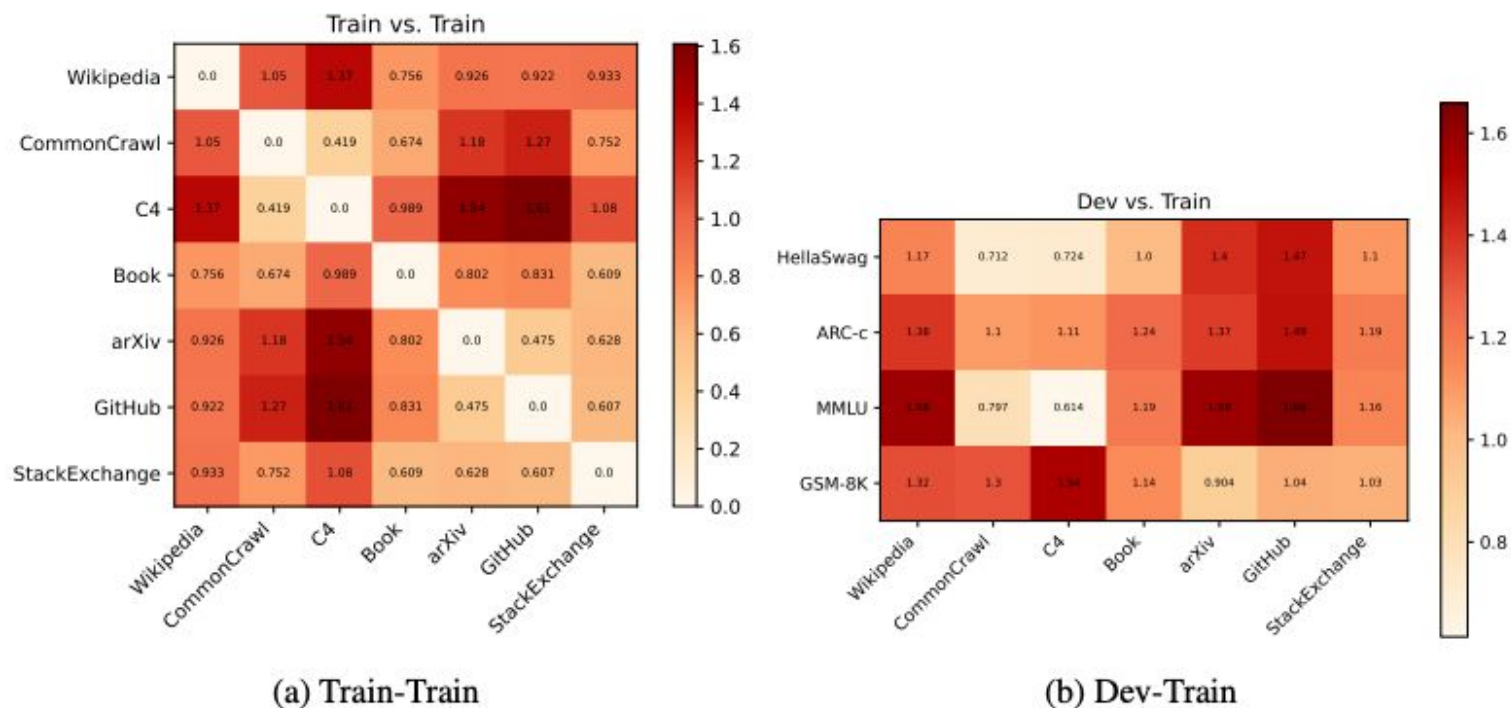
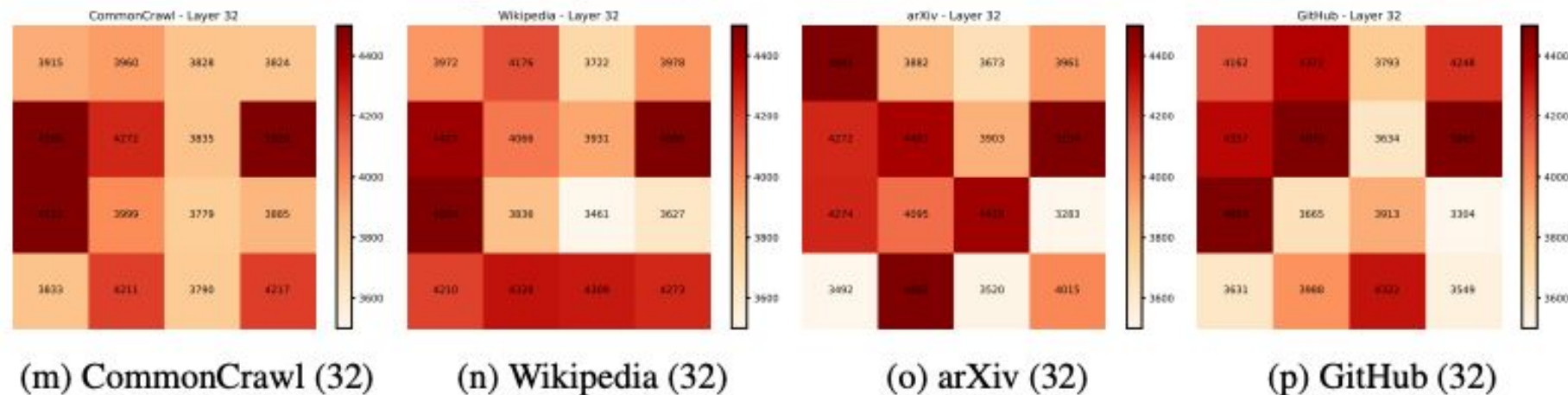


Expert Specialization

Expert	Top-5 preceding tokens
5	year, years, billion, millions, tonnes
9	electronic, local, public, national, outdoor
34	to, will, should it, may
42	two, 50, 1, 80, 000
62	work, started, involved, working, launched
72	is, was, be, been, were
74	going, go, come, back, return
101	B, T, W, H, k

Table 2: **Expert specialization based on preceding context in BASE Layers.** We reproduce a portion of table of [Lewis et al. \(2021\)](#), presenting the most frequent preceding top-five tokens for the selected experts. This example shows experts specializing in punctuation, conjunctions & articles, verbs, visual descriptions, proper names, counting & numbers.

Expert Specialization



Thanks

Q&A

Prof. Yu Cheng
chengyu@cse.cuhk.edu.hk

